



Trabalho de Conclusão de Curso

Controle determinístico para CNC baseado em STM32L475 com DDA de alta frequência

de Valdir Dias Silva Junior

orientado por

Prof. Dr. Ícaro Bezerra Queiroz de Araújo

Universidade Federal de Alagoas
Instituto de Computação
Maceió, Alagoas
12 de Novembro de 2024

UNIVERSIDADE FEDERAL DE ALAGOAS
Instituto de Computação

CONTROLE DETERMINÍSTICO PARA CNC BASEADO EM STM32L475 COM DDA DE ALTA FREQUÊNCIA

Trabalho de Conclusão de Curso submetido
ao Instituto de Computação da Universidade
Federal de Alagoas como requisito parcial
para a obtenção do grau de Engenheiro de
Computação.

Valdir Dias Silva Junior

Orientador: Prof. Dr. Ícaro Bezerra Queiroz de Araújo

Banca Avaliadora:

Hugo	Profª. Dra., IFAL
Andressa	MSc., SENAI CIMATEC

Maceió, Alagoas
12 de Novembro de 2024

Dados para elaboração da ficha catalográfica devem ser inseridos aqui.
Substitua este texto pelo conteúdo oficial fornecido pela biblioteca ou órgão responsável.

Ata de Aprovação

Pagina destinada a ata da banca examinadora

Dedicatória

Conteúdo pendente

Agradecimentos

Conteudo pendente

Epígrafe

Conteudo pendente

Resumo

Este trabalho descreve o desenvolvimento de um controlador CNC com foco em determinismo temporal, implementado sobre o microcontrolador STM32L475 e integrado a uma Raspberry Pi. O projeto emprega o gerador de pulsos baseado em *Digital Differential Analyzer* (DDA) a 50 kHz usando o temporizador TIM6, fecha o laço PID de 1 kHz com o TIM7 e utiliza encoders em modo quadratura nos timers TIM2, TIM3 e TIM5 para estimar posição. A comunicação com o host segue um protocolo SPI com DMA circular, complementado por logs via USART1, garantindo observabilidade sem comprometer os prazos de tempo real.

A fundamentação teórica apresenta conceitos de CNC, modelagem de motores de passo, controle PID e algoritmos DDA. A metodologia foi conduzida por um roteiro incremental que validou clock, interrupções críticas, laços de controle e serviços de comunicação. Os resultados mostram a estabilidade do DDA em 50 kHz, variação temporal (*jitter*) inferior a $3\mu\text{s}$ no laço PID e o comportamento do pipeline SPI, destacando a necessidade de três ciclos de enqueue para o *ack* de comandos de LED. O trabalho conclui evidenciando a robustez do firmware proposto e apresenta sugestões para otimização futura do DMA e dos serviços auxiliares.

Palavras-chave: controle CNC; STM32L475; DDA; controlador PID; SPI determinístico.

Abstract

This dissertation describes the development of a CNC controller focused on temporal determinism, implemented on the STM32L475 microcontroller and integrated with a Raspberry Pi host. The design uses a 50 kHz pulse generator based on a Digital Differential Analyzer (DDA) running on timer TIM6, closes a 1 kHz PID loop with TIM7, and leverages quadrature encoders through timers TIM2, TIM3, and TIM5 for position estimation. Communication with the host is handled by an SPI protocol with circular DMA, complemented by USART1 logging to preserve observability without affecting real-time constraints.

The theoretical background covers CNC systems, stepper motor modeling, PID control, and DDA algorithms. The methodology followed an incremental bring-up roadmap that validated the clock tree, critical interrupts, control loops, and communication services. Results show stable DDA output at 50 kHz, PID loop jitter below 3 μ s, and the behavior of the SPI poll pipeline, highlighting the need for three cycles to acknowledge LED commands. The work concludes by emphasizing the robustness of the proposed firmware and suggests future optimizations for DMA handling and auxiliary services.

Keywords: CNC control; STM32L475; DDA; PID controller; deterministic SPI.

Lista de Figuras

3.1	Fluxo do comando <code>cnc-cli</code> : validação, aplicação de patches TMC, enfileiramento de movimentos no STM32, monitoramento e estados do LED. . .	20
3.2	Sobreposição entre velocidade medida e FOPDT (eixo X).	23
3.3	Sobreposição entre velocidade medida e FOPDT (eixo Y).	23
3.4	Sobreposição entre velocidade medida e FOPDT (eixo Z).	23
3.5	Modelo 3D do controlador CNC com placa STM32, interface para Raspberry Pi, base de prototipagem e módulos de potência TMC5160.	24
3.6	Modelo 3D do suporte do encoder TMCS-28 e do ponteiro acoplado ao eixo do motor de passo.	26
3.7	Vista dos motores de passo com suportes impressos em 3D, ponteiros e encoder TMCS-28 acoplado ao eixo para realimentação de posição.	26
3.8	Protótipo do controlador CNC montado em bancada, com fonte de alimentação, motores de passo, drivers TMC5160, placa STM32 e Raspberry Pi.	28
3.9	Comparação visual entre o pulso de STEP implementado no firmware (esquerda) e o pulso mínimo requerido pelo datasheet do TMC5160 (direita), evidenciando a robusta margem de tempo utilizada.	29
3.10	Simulação da operação interna do DDA para um movimento de 7 passos em X e 4 em Y. O gráfico superior mostra a evolução dos acumuladores de fase, e o inferior mostra os pulsos STEP gerados para cada eixo a cada transbordamento.	31
3.11	Perfil de velocidade da rampa trapezoidal simulado a partir dos parâmetros do firmware.	35
3.12	Cenário de simulação com atrito programável por eixo, destacando as janelas de carga e o impacto sobre as curvas de posição, velocidade e erro de sincronismo.	39

Lista de Tabelas

2.1	Layout do quadro SPI (tamanho fixo: 42 bytes).	9
2.2	Requisição LED (CMD=0x10).	9
2.3	Resposta LED.	9
2.4	Requisição Movimento (CMD=0x20).	10
2.5	Resposta Movimento.	10
2.6	Quadro de escrita TMC5160 (5 bytes, big-endian).	11
2.7	Quadro de leitura TMC5160 (5 bytes, big-endian; resposta no ciclo seguinte).	11
2.8	Exemplos de acessos SPI ao TMC5160.	12
2.9	Estimativa da corrente mínima prática (I_{RMS}).	15
2.10	Registros STM32 L4 utilizados neste trabalho (ver [STMicroelectronics, 2013, STMicroelectronics, 2018a]).	16
2.11	Registros TMC5160 utilizados neste trabalho (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]).	17
3.1	Comparativo entre tempos mínimos do TMC5160 e tempos configurados no firmware.	29
3.2	Parâmetros de tempo e limites usados no firmware.	36

Lista de Símbolos

- f_{sys} Frequência do clock principal do microcontrolador.
- f_{TIM6} Frequência de interrupção do temporizador TIM6 usada pelo DDA.
- f_{loop} Frequência do laço de controle executado no TIM7.
- N_{steps} Número de pulsos STEP gerados para cada eixo.
- p Contagem acumulada de pulsos do atuador (STEP).
- c Contagem acumulada do encoder (em contagens).
- k Índice discreto de amostragem ($k \in \mathbb{N}$), com $t = k T_s$.
- $f(k)$ Valor da grandeza discreta f na k -ésima amostra.
- $v_{\text{cmd}}(k) / V_{\text{cmd}}$ Velocidade de comando: $v_{\text{cmd}}(k)$ é o valor instantâneo (pulsos STEP/s) na amostra k ; V_{cmd} denota a amplitude do degrau de comando utilizada nas análises (tipicamente $V_{\text{cmd}} := \bar{v}_{\text{cmd}}$).
- θ Posição angular estimada a partir do encoder.
- $e(t)$ Erro instantâneo entre referência e posição medida.
- $u(t)$ Sinal de controle produzido pelo regulador PID.
- K_p, K_i, K_d Ganhos proporcional, integral e derivativo do controlador.
- J Momento de inércia equivalente do eixo controlado.
- T_L Torque de carga refletido no eixo do motor de passo.
- T_s Período de amostragem do laço de controle.
- V_{bus} Tensão do barramento que alimenta os drivers TMC5160.

Lista de Abreviaturas

CNC Controle Numérico Computadorizado.

DDA *Digital Differential Analyzer.*

DMA *Direct Memory Access.*

GPIO *General Purpose Input/Output.*

NVIC *Nested Vectored Interrupt Controller.*

PID Proporcional-Integral-Derivativo.

SOF *Start of Frame* (Início do quadro; marcador de início em protocolos de quadro).

EOF *End of Frame* (Fim do quadro; marcador de encerramento em protocolos de quadro).

STEP/DIR/EN Sinais de passo, direção e habilitação dos drivers de motor de passo.

USART *Universal Synchronous/Asynchronous Receiver/Transmitter.*

VCP *Virtual COM Port.*

UFAL Universidade Federal de Alagoas.

Conteúdo

1	Introdução	1
1.1	Objetivos	1
1.1.1	Objetivo Geral	1
1.1.2	Objetivos Específicos	2
1.2	Organização do texto	2
2	Fundamentação Teórica	3
2.1	Sistemas CNC	3
2.2	Temporizadores do STM32L475	3
2.3	Digital Differential Analyzer	4
2.4	Modelagem de motores de passo	4
2.5	Controlador PID digital	5
2.6	Modelo FOPDT e síntese de ganhos PD	5
2.6.1	Derivação da resposta ao degrau	5
2.7	Integração PID-DDA	6
2.8	Sincronização Cruzada de Eixos	7
2.9	Comunicação SPI e USART	8
2.10	Driver Trinamic TMC5160 e Encoder TMCS-28	11
2.10.1	TMC5160	11
2.10.2	TMCS-28 (Trinamic/Analog Devices)	12
2.10.3	Implicações de projeto	12
2.11	Motores de Passo NEMA 23, Passo de 0,9° e Seleção da Corrente I_{RMS}	13
2.11.1	NEMA 23: geometria e implicações mecânicas	13
2.11.2	Passo elétrico: 1,8° vs 0,9°	13
2.11.3	Microstepping, suavidade e taxa de passos	13
2.11.4	Parâmetros elétricos e modelo de primeira ordem	14
2.11.5	Torques relevantes	14
2.11.6	Resumo para os motores usados	14
2.11.7	Fundamentação teórica para a seleção de I_{RMS}	14
2.12	Registros de Configuração Utilizados	15
2.12.1	STM32L475	16

2.12.2	TMC5160	17
3	Metodologia	18
3.1	Roteiro incremental de inicialização	18
3.2	Arquitetura de software	18
3.2.1	Fluxo do comando <code>cnc-cli</code>	19
3.3	Procedimentos de teste	21
3.4	Identificação experimental do modelo FOPDT	21
3.4.1	Pré-processamento e séries derivadas	22
3.4.2	Extração de marcos temporais	22
3.4.3	Estimativas K , L e τ	22
3.4.4	Síntese PD e validação	22
3.4.5	Validação gráfica do modelo FOPDT	23
3.5	Arquitetura de Hardware	23
3.5.1	Visão geral do sistema	24
3.5.2	Alimentação e EMC	24
3.5.3	Drivers TMC5160	24
3.5.4	Encoder óptico TMCS-28	25
3.5.5	Intertravamentos e segurança	26
3.5.6	PCB, cabeamento e montagem	27
3.6	Arquitetura de Controle de Movimento (Visão Geral)	28
3.6.1	Compatibilidade de <i>timing</i> com o TMC5160 (<i>STEP/DIR</i>)	28
3.6.2	MicroPlyer, resolução e DEDGE	29
3.7	DDA em Ponto Fixo (TIM6 @ 50 kHz)	30
3.8	Rampa Trapezoidal (TIM7 @ 1 kHz)	33
3.9	Controle PI de Posição com Encoder	35
3.10	Fila de Movimentos e Segmentação	35
3.11	Mapeamento para o TMC5160	36
3.11.1	Geração de STEP/DIR (<i>"PWM" de passo</i>)	36
3.11.2	Resolução de microstepping e MicroPlyer	36
3.12	Parâmetros-chaves do Projeto	36
3.13	Coordenação Multi-eixos	36
3.14	Simulador Interativo em Python	37
3.15	Boas práticas e Diagnóstico	39
4	Resultados e Discussão	40
4.1	Desempenho dos serviços principais	40
4.2	Análise do pipeline SPI	40
5	Conclusão	41

Capítulo 1

Introdução

A popularização de máquinas de Controle Numérico Computadorizado (CNC) depende de controladores capazes de converter instruções digitais em movimentos sincronizados com elevado grau de previsibilidade. No contexto de manufatura de pequeno e médio porte, soluções baseadas em computadores pessoais apresentam custo acessível, mas sofrem com a variabilidade de latência inerente aos sistemas operacionais de propósito geral. Este trabalho investiga uma alternativa embarcada utilizando o microcontrolador STM32L475, capaz de operar a 80 MHz e oferecer temporizadores avançados para geração de pulsos e amostragem de dados [STMicroelectronics, 2013]. Ao combinar a unidade embarcada com uma Raspberry Pi responsável pela interface de alto nível, busca-se garantir escalabilidade e facilidade de integração com pipelines de produção.

A motivação está ligada ao desenvolvimento de um controlador determinista que gere pulsos STEP/DIR/EN em 50 kHz, execute o laço PID a 1 kHz e mantenha comunicação confiável com o host via SPI e USART, entregando registros de telemetria para diagnósticos. A arquitetura proposta precisa coordenar três eixos de motores de passo monitorados por encoders de alta resolução, sincronizar serviços de homing e segurança, e permitir a inserção de novas rotinas sem comprometer as janelas temporais estabelecidas.

1.1 Objetivos

1.1.1 Objetivo Geral

Projetar e documentar um controlador CNC determinístico baseado no STM32L475, integrando geração DDA de pulsos, controle PID em tempo real e protocolo de comunicação SPI com um cliente Raspberry Pi.

1.1.2 Objetivos Específicos

- Configurar o temporizador TIM6 para gerar pulsos em 50 kHz utilizando o método DDA.
- Implementar o loop de controle PID a 1 kHz no TIM7, incorporando leitura incremental dos encoders.
- Estruturar o firmware em camadas modulares (Core, App e Services) que facilitem a validação incremental.
- Validar o pipeline de comunicação SPI escravo com DMA circular e logs assíncronos via USART1.
- Registrar testes que comprovem baixa variação temporal (*jitter*) e estabilidade nos serviços críticos.

1.2 Organização do texto

O Capítulo 2 apresenta a fundamentação teórica sobre CNC, temporizadores STM32, DDA, controle PID e protocolos de comunicação. O Capítulo 3 descreve a metodologia de inicialização incremental utilizada para configurar o firmware e o hardware de suporte. Em seguida, o Capítulo 4 reúne os resultados experimentais, analisando desempenho dos serviços e o comportamento do pipeline SPI. Por fim, o Capítulo 5 sintetiza as contribuições, limitações e linhas futuras de trabalho.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta os conceitos fundamentais utilizados na implementação do controlador CNC embarcado. São abordados os princípios de sistemas CNC, a configuração de temporizadores no STM32L475, os métodos DDA para geração de pulsos, a modelagem de motores de passo, os controladores PID e os aspectos de comunicação SPI/USART empregados no projeto.

2.1 Sistemas CNC

Máquinas de Controle Numérico Computadorizado interpretam instruções codificadas (por exemplo, G-code) e convertem essas ordens em movimentos coordenados entre múltiplos eixos, garantindo precisão repetível no processo de usinagem ou manufatura [Groover, 2015]. Controladores modernos combinam processamento em tempo real com interfaces de alto nível para planejar trajetórias, compensar erros e monitorar a execução. A adoção de arquiteturas híbridas—com uma unidade embarcada responsável pelo tempo real e um computador auxiliar para supervisão—diminui a suscetibilidade a variações temporais (*jitter*) e a perdas de sincronismo, mantendo a flexibilidade de integração com sistemas de gestão.

2.2 Temporizadores do STM32L475

O microcontrolador STM32L475 disponibiliza temporizadores de uso geral e avançado capazes de operar na faixa de 80 MHz com alta resolução temporal. A configuração de um temporizador baseia-se na relação

$$f_{\text{TIM}} = \frac{f_{\text{bus}}}{(PSC + 1)(ARR + 1)}, \quad (2.1)$$

onde f_{bus} representa a frequência do barramento APB, PSC o prescaler e ARR o registrador de auto-reload. O guia de aplicação oficial descreve como esses parâmetros possibilitam gerar bases de tempo precisas para laços de controle, captura de entradas e geração de pulsos PWM [STMicroelectronics, 2013]. No projeto, o TIM6 é configurado com $PSC = 79$ e $ARR = 19$ para obter interrupções em 50 kHz, enquanto o TIM7 opera com $PSC = 7999$ e $ARR = 9$, produzindo um laço de 1 kHz. Os temporizadores TIM2, TIM3 e TIM5 operam em modo encoder para rastrear incrementalmente a posição dos eixos.

2.3 Digital Differential Analyzer

Algoritmos DDA são integradores digitais que aproximam trajetórias contínuas por meio de incrementos discretos, amplamente utilizados em sistemas de gráficos e em controladores de movimento para motores de passo [Fussell, 2003]. O método acumula um erro fracionário em cada interação e emite um pulso quando a soma ultrapassa um limiar definido, resultando em uma sequência de passos que aproxima a velocidade ou a trajetória desejada. Arquiteturas clássicas de interpolação tratam o DDA como núcleo do gerador de pulsos, responsável por alimentar os laços de posição e velocidade que seguem a referência calculada amostra a amostra [IDC Technologies, 2014, Koren, 1978, Wang and Hu, 2016, Suh et al., 2008]. Panoramas comparativos mostram como variantes circular, linear e por superfície mantêm avanço constante mesmo em trajetórias multi-eixo, o que fundamenta a escolha de incrementos acumulativos sincronizados para o firmware do STM32 [Koren, 2010]. No contexto deste trabalho, o DDA implementado no TIM6 gera sinais STEP com resolução de 20 μs e tolera ajustes de velocidade em tempo real sem quebrar a coesão dos múltiplos eixos. A abordagem permite sincronizar movimentos lineares e circulares através da atualização de incrementos acumulados por eixo durante a ISR do temporizador, preservando a planicidade de avanço descrita pelas referências.

2.4 Modelagem de motores de passo

Motores de passo híbridos apresentam dinâmica eletromecânica dominada por indutâncias de fase, resistência de enrolamento e um torque relacionado à diferença angular entre rotor e campo magnético. Modelos clássicos de motores de passo descrevem a relação entre corrente, torque e velocidade angular por meio de equações diferenciais acopladas que podem ser discretizadas para implementação em controladores digitais [Kenjo and Sugawara, 1994]. A precisão do controle depende da estimação do torque de carga T_L , do momento de inércia equivalente J e da compensação de efeitos como ressonâncias de meia etapa. A utilização de encoders em modo quadratura provê realimentação adicional para compensar perda de passos e acúmulo de erro estático.

2.5 Controlador PID digital

O controlador Proporcional-Integral-Derivativo (PID) continua sendo uma das estratégias mais difundidas para controle de processos, combinando uma ação proporcional que reage ao erro instantâneo, um termo integral que remove erro estacionário e um termo derivativo que prevê tendências de variação [Åström and Hägglund, 1995]. Loops servo em máquinas CNC seguem as posições discretizadas pelo interpolador e corrigem desvios com ganhos sintonizados para cada eixo, podendo incluir observadores ou compensação de distúrbios para manter a precisão em alta velocidade [Wang and Hu, 2016, Koren, 1980]. Para implementação digital, é comum utilizar a forma incremental

$$u[k] = u[k-1] + K_p(e[k] - e[k-1]) + K_i T_s e[k] + \frac{K_d}{T_s}(e[k] - 2e[k-1] + e[k-2]), \quad (2.2)$$

onde T_s é o período de amostragem. No laço de 1 kHz, o período fixo reduz o esforço computacional e facilita a análise de estabilidade. Estratégias derivadas, como anti-*windup* e filtros de primeira ordem no termo derivativo, são essenciais para lidar com o ruído proveniente dos encoders e das variações do torque de carga. Pesquisas recentes destacam ainda controladores PID acoplados/cross-coupled que tratam o erro de contorno entre eixos como variável adicional, reduzindo desvios em trajetórias complexas [Wang et al., 2021].

2.6 Modelo FOPDT e síntese de ganhos PD

O comportamento dinâmico da malha de velocidade foi aproximado por um modelo de primeira ordem com atraso puro (FOPDT, do inglês *First-Order Plus Dead Time*), cuja função de transferência é dada por

$$G_v(s) = \frac{K e^{-Ls}}{\tau s + 1}, \quad (2.3)$$

onde K denota o ganho estático, L o tempo morto e τ a constante de tempo. Para um degrau de amplitude u_0 , a resposta temporal é

$$y(t) = \begin{cases} 0, & t < L, \\ K u_0 \left(1 - e^{-\frac{t-L}{\tau}}\right), & t \geq L. \end{cases} \quad (2.4)$$

Propriedades úteis incluem $t_{63} = L + \tau$ (63,2% do valor de regime) e $t_{90} \approx L + 2,303 \tau$.

2.6.1 Derivação da resposta ao degrau

Considere o modelo de primeira ordem com atraso puro excitado por um degrau de amplitude U atrasado de L segundos, isto é, $u(t) = U H(t - L)$, onde $H(\cdot)$ é a função de

Heaviside. No domínio do tempo, o FOPDT satisfaz a equação diferencial linear

$$\tau \dot{y}(t) + y(t) = K u(t) = K U H(t - L), \quad y(0) = 0. \quad (2.5)$$

Para $t < L$ tem-se $u(t) = 0$ e, portanto, $y(t) = 0$. Para $t \geq L$ define-se $\xi = t - L$ e resolve-se o problema deslocado

$$\tau \frac{d}{d\xi} y(L + \xi) + y(L + \xi) = K U, \quad y(L^-) = 0. \quad (2.6)$$

A solução geral é soma das soluções homogênea e particular, resultando em

$$y(t) = K U (1 - e^{-\frac{t-L}{\tau}}) H(t - L), \quad (2.7)$$

que coincide com a expressão utilizada nas validações: para $t < L$, $y = 0$; para $t \geq L$, a saída converge exponencialmente para $K U$ com constante de tempo τ .

Uma dedução equivalente via transformada de Laplace parte de $G_v(s) = \frac{K e^{-Ls}}{\tau s + 1}$ e de um degrau de magnitude U , U/s . Pela propriedade de translação temporal, $Y(s) = G_v(s) \frac{U}{s} = \frac{K U}{s(\tau s + 1)} e^{-Ls}$, cuja transformada inversa fornece a mesma expressão $y(t) = K U (1 - e^{-(t-L)/\tau}) H(t - L)$.

No contexto deste trabalho, a amplitude do degrau U é escolhida como a velocidade de comando em regime (denotada por V_{cmd}), $U := V_{\text{cmd}} := \bar{v}_{\text{cmd}}$, estimada a partir da média em janela final do traço de comando (vide Seção 3.4). Essa escolha assegura que o nível de regime previsto pelo modelo, $K U$, coincida com o patamar medido no encoder quando a aproximação FOPDT é adequada.

Com base nesse modelo, adotou-se um regulador proporcional-derivativo (PD) na malha de velocidade, mantendo $K_i = 0$ por já existir ação integrativa no nível de posição. As constantes são sintetizadas pelas regras de Ziegler-Nichols para curva de reação (processos do tipo FOPDT), a saber [Åström and Hägglund, 1995]:

$$K_p = 1,2 \frac{\tau}{K L}, \quad T_d = 0,5 L, \quad K_d = K_p T_d, \quad K_i = 0. \quad (2.8)$$

Para implementação digital, os ganhos podem ser escalonados para representação inteira utilizando um fator de escala $S > 0$ (*fixed-point*), *e.g.*, $k_{p,i} = \text{round}(S K_p)$ e análogos para $k_{i,i}$ e $k_{d,i}$. Tal mapeamento preserva a coerência entre a análise contínua e o controlador discreto executado em tempo real.

2.7 Integração PID-DDA

A sincronização entre o DDA e o controlador PID ocorre ao transformar o comando de posição desejada em incrementos de passos por período do TIM6. Métodos de dis-

tribuição diferencial garantem que ajustes produzidos pelo PID sejam refletidos sem rupturas na geração de pulsos, mantendo o alinhamento entre eixos [Mori et al., 2005, Wang and Hu, 2016]. Materiais didáticos e relatórios industriais ilustram o fluxo completo: o interpolador entrega referências de posição, o encoder fornece o retorno real e o PID calcula o esforço aplicado ao motor para cancelar o erro, em um ciclo repetido a cada 1 ms [IDC Technologies, 2014, Suh et al., 2008]. Implementações modernas combinam esses blocos em FPGAs ou SoCs para reduzir latência e habilitar interpolação multi-eixo síncrona com laços servo dedicados [Koren, 1980]. No firmware do STM32, o laço de controle alimenta as metas de velocidade e microstepping de cada eixo, enquanto o DDA executa as transições discretas, possibilitando perfis suaves e respeitando os limites de aceleração definidos pelo firmware.

2.8 Sincronização Cruzada de Eixos

Em movimentos multi-eixo, a coordenação cinemática exige que cada eixo siga uma fração coerente do avanço global, minimizando erro de sincronismo e de contorno. Neste texto, adotamos o termo *Sincronização Cruzada de Eixos* para englobar o conjunto de estratégias que, sobre a malha de posição/velocidade por eixo, impõem coerência temporal do grupo, em linha com abordagens de controle *cross-coupled* de contorno descritas em [Wang and Hu, 2016, Koren, 1980, Koren, 2010].

Sejam $s_i(t)$ as posições desejadas e $x_i(t)$ as posições medidas. O *progresso normalizado* por eixo pode ser escrito como

$$p_i(t) = \frac{x_i(t) - x_i(0)}{\max\{1, |s_i(T) - s_i(0)|\}}, \quad (2.9)$$

onde T é a duração nominal do movimento. Uma escolha comum de eixo *mestre* é aquele com maior *restante absoluto* $R_i(t) = |s_i(T) - x_i(t)|$ (ou, inversamente, menor progresso); o objetivo é que os demais eixos acompanhem o mestre, reduzindo o erro de sincronismo

$$e_i^{\text{sync}}(t) = p_m(t) - p_i(t), \quad i \neq m. \quad (2.10)$$

Uma lei simples de *modulação de avanço* aplica um reescalonamento na velocidade de referência de cada eixo não-mestre, do tipo

$$v_i^{\text{ref}}(t) = \sigma(e_i^{\text{sync}}(t)) v_i^{\text{nom}}(t), \quad \sigma(e) = 1 - (1 - \sigma_{\min}) \min\left\{1, \frac{|e|}{e_{\text{th}}}\right\}, \quad (2.11)$$

com $\sigma_{\min} \in (0, 1)$ a fração mínima admissível e $e_{\text{th}} > 0$ um limiar que define quando a penalização satura. Intuitivamente, quanto maior o atraso relativo e_i^{sync} , maior a redução da velocidade "instantânea" para permitir a re-sincronização. Em situações críticas, uma

retenção de sincronismo (*sync hold*) pode impor $v_i^{\text{ref}}=0$ para eixos defasados além de um limite de segurança.

Durante a fase final do movimento, define-se uma *janela de acabamento* de largura W (em passos ou fração do deslocamento global), na qual se aplicam regras mais conservadoras: desabilita-se detecção de *stall*, eleva-se $\sigma_{\min}$ e relaxam-se as trocas de mestre para evitar oscilações. Tais práticas são consistentes com técnicas de controle cruzado de eixos *cross-coupled* que utilizam o erro de contorno/ sincronismo para melhorar a precisão em trajetórias 2D/3D [Wang et al., 2021].

Note-se que a sincronização cruzada não substitui as malhas PID por eixo: ela opera como supervisor de alto nível, gerando referências coerentes e respeitando limitações de aceleração e saturação. A estabilidade do conjunto depende da escolha de limiares (e_{th} , $\sigma_{\min}$) e da política de seleção do mestre (maior restante, menor progresso, eixo carregado etc.), que devem ser ajustados de modo a preservar margens de fase e evitar comandos abruptos.

2.9 Comunicação SPI e USART

A comunicação com a Raspberry Pi utiliza o periférico SPI1 em modo escravo com DMA circular. Esse desenho reduz a carga da CPU e garante que as transferências de 42 bytes sejam executadas dentro da janela entre interrupções do TIM6 e TIM7. A USART1, configurada como *Virtual COM Port*, é utilizada para depuração e registro de eventos críticos, com uma fila não bloqueante que impede o impacto sobre o laço de controle. A coordenação entre SPI e USART é fundamental para evitar inversões de prioridade que poderiam comprometer o determinismo do sistema [STMicroelectronics, 2018b]. A análise do pipeline de polls evidencia como o firmware pausa e reinicia o DMA para garantir que cada resposta seja transmitida somente após processamento completo.

SPI Raspberry Pi ↔ STM32: protocolo do enlace

O enlace SPI escravo (STM32 como escravo) utiliza quadros de tamanho fixo com **42 bytes**, transferidos em modo *full-duplex*. O mestre (Raspberry Pi) envia um *poll* com a requisição e recebe, no mesmo ciclo ou nos ciclos subsequentes, a resposta preparada pelo firmware (via fila de respostas e DMA). O quadro adota marcadores de início/fim para robustez:

Quando não há resposta pronta, o escravo devolve um quadro com todos os 21 bytes preenchido com `0xA5`. Quando a resposta de um serviço está completa, o escravo publica o quadro final com `E0F=0x54` e CRC válido, conforme observado no cliente `cnc_spi_client.py`.

Tabela 2.1: Layout do quadro SPI (tamanho fixo: 42 bytes).

Offset	Tamanho	Campo	Descrição
0	1	SOF	Byte fixo de início (0xAB) para sincronização do quadro.
1	1	CMD	Identificador do serviço/comando (ex.: 0x10=LED, 0x20=MOVE).
2	1	FLAGS	Bits de controle/opções do comando (reservado = 0x00).
3	1	LEN	Comprimento válido do PAYLOAD (0 a 34 bytes).
4	34	PAYLOAD	Dados da requisição; preencher com 0x00 quando não usado.
38	2	CRC16	Verificação de integridade (LSB,MSB) calculada sobre bytes 0..37.
40	1	RSV	Reservado (0x00) para alinhamento/expansão futura.
41	1	EOF	Marcador de fim: 0x54.

Serviço LED (exemplo de comando simples)

O serviço LED liga/desliga ou alterna o estado. O mestre envia uma requisição; a resposta confirma o estado final.

Tabela 2.2: Requisição LED (CMD=0x10).

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x10 (LED)
2	1	FLAGS	0x00 (padrão)
3	1	LEN	0x02
4	1	LED_ID	0x00 (LED on-board)
5	1	MODE	0x00=OFF, 0x01=ON, 0x02=TOGGLE
6..37	32	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Tabela 2.3: Resposta LED.

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x10 (eco)
2	1	FLAGS	0x00
3	1	LEN	0x02
4	1	STATUS	0x00=OK, >0=erro
5	1	LED_STATE	0x00=OFF, 0x01=ON
6..37	32	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Serviço Movimento (exemplo de comando composto)

O serviço de movimento recebe metas por eixo e parâmetros cinemáticos simplificados. Campos não usados devem ser zero.

Tabela 2.4: Requisição Movimento (CMD=0x20).

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x20 (MOVE)
2	1	FLAGS	0x00
3	1	LEN	Deve refletir o total de bytes uteis do payload (exemplo:0x15).
4	1	AXIS_MASK	Máscara de eixos: Bit0=X, Bit1=Y, Bit2=Z. Permite ativar eixos individualmente.
5..8	4	X_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
9..12	4	Y_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
13..16	4	Z_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
17..20	4	FEED	Velocidade máxima alvo em passos/s (uint32). Limitador para o perfil de movimento.
21	1	JERK	Parâmetro opcional do perfil (suavização). Se não utilizado, enviar 0x00.
22..37	16	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Tabela 2.5: Resposta Movimento.

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x20 (eco)
2	1	FLAGS	0x00
3	1	LEN	0x03
4	1	STATUS	0x00=aceito, 0x01=ocupado, >0=erro
5	1	QUEUE_DEPTH	Itens pendentes na fila de movimentos
6	1	RESERVED	0x00
7..37	31	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Notas sobre os campos da resposta MOVE:

- **STATUS:** 0x00=aceito (comando enfileirado/executando); 0x01=ocupado (tentar novamente); valores >0 indicam erro (cdigo específico do firmware).
- **QUEUE_DEPTH:** tamanho atual da fila de movimentos, til para controle de fluxo do mestre.
- **CRC16:** valida a integridade do quadro; CRC invlido deve ser tratado como erro de comunicao.

SPI Raspberry Pi ↔ TMC5160: configuração por registradores

O TMC5160 expõe um barramento SPI para configuração e diagnóstico por meio de transações de **40 bits** (5 bytes por quadro), compostas por um byte de endereço e quatro bytes de dados. O bit mais significativo do byte de endereço indica leitura/escrita (**1**=leitura, **0**=escrita) e os 7 bits menos significativos selecionam o registrador. O protocolo apresenta **latência de uma transação**: os dados retornados em *SD0* referem-se ao comando emitido no quadro anterior. Recomenda-se emitir uma leitura “de preparo” antes de coletar o valor válido do registrador desejado (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]). Em implementações usuais o SPI opera em modo 3 (CPOL=1, CPHA=1).

Tabela 2.6: Quadro de escrita TMC5160 (5 bytes, big-endian).

Offset	Tamanho	Campo	Descrição
0	1	ADDR[6:0], R/W=0	Endereço do registrador (7 bits), bit 7=0 (escrita).
1..4	4	DATA[31:0]	Palavra de 32 bits, ordem de bytes: MSB → LSB.

Tabela 2.7: Quadro de leitura TMC5160 (5 bytes, big-endian; resposta no ciclo seguinte).

Offset	Tamanho	Campo	Descrição
0	1	ADDR[6:0], R/W=1	Endereço do registrador (7 bits), bit 7=1 (leitura).
1..4	4	DUMMY	0x00; os 32 bits lidos pertencem ao quadro anterior.

Exemplos de registros comuns na bring-up e operação são apresentados junto ao TMC5160 (Seção 2.10.1).

Os exemplos acima alinham-se ao fluxo de *poll* descrito na Seção 2.7.

2.10 Driver Trinamic TMC5160 e Encoder TMCS-28

Os drivers de motor de passo da Trinamic são amplamente utilizados por oferecerem modos de comutação silenciosos, detecção de carga *sensorless* e parametrização fina via registradores. Em complemento, encoders ópticos incrementais como o TMCS-28 permitem medição sem contato da posição/ângulo do eixo com disco óptico e sensor, habilitando calibrações de zero, telemetria e malhas fechadas. Esta seção resume os recursos relevantes do TMC5160 e do TMCS-28 e como eles se relacionam à arquitetura do firmware.

2.10.1 TMC5160

O TMC5160 é um driver de alto desempenho voltado a correntes elevadas, com interface SPI e controle por pinos STEP/DIR/EN. Entre os principais recursos:

- **Microstepping e microPlyer**: geração de até 256 micropassos e interpolação *microPlyer* a partir de entradas com baixa resolução, suavizando o movimento mesmo com taxas de pulso moderadas.
- **stealthChop2**: modo de chaveamento focado em baixo ruído acústico; configurado principalmente em PWMCONF.
- **spreadCycle**: chopper clássico de corrente para maior fidelidade em torque em altas velocidades; parametrização em CHOPCONF.
- **StallGuard2**: medição de carga sem sensor (*sensorless*) que permite detectar perda de passo/contato; limiar em SGTHRS e leitura do ganho em SG_RESULT.
- **coolStep**: regulação dinâmica de corrente baseada na carga para reduzir perdas sem comprometer torque.

dcStep: avanço dependente de carga para evitar perda de passo em condições adversas. - **Proteções e diagnóstico:** DRV_STATUS expõe flags de sobrecorrente, subtensão e temperatura.

Na integração com o firmware, o TMC5160 é inicializado via SPI ajustando IHOLD_IRUN (correntes de hold/run), TPOWERDOWN, CHOPCONF e PWMCONF. A comutação de perfis (*stealthChop2* $\leftrightarrow$ *spreadCycle*) pode ser feita em tempo de execução para equilibrar ruído e robustez. Para homing *sensorless*, calibra-se SGTHRS e a janela TCOOLTHRS de maneira a obter detecção repetível sem falsos positivos.

Tabela 2.8: Exemplos de acessos SPI ao TMC5160.

Registro	Operação/Observação
IHOLD_IRUN	Escrita dos campos IHOLD, IRUN e IHOLDDELAY para corrente de hold/run e rampa.
CHOPCONF	Seleção de <i>spreadCycle</i> ou parâmetros de histerese do chopper.
PWMCONF	Parâmetros do <i>stealthChop2</i> (PWM_AMPL, PWM_GRAD).
SGTHRS	Limiar do <i>StallGuard2</i> para homing sensorless.
DRV_STATUS	Leitura de flags de falha e SG_RESULT; lembrar da latência de um quadro.

2.10.2 TMCS-28 (Trinamic/Analog Devices)

O TMCS-28 é um encoder óptico incremental de baixo custo e dimensão reduzida para motores de passo e PMSM/BLDC. Utiliza roda código óptica e fornece saídas em quadratura **A** e **B** mais **N** (*index*). Principais pontos ([TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), 2022]):

- **Resoluções:** até 625 lpr (*lines per rotation*) $\Rightarrow$ 40 000 cpr; opção de 64 lpr $\Rightarrow$ 4 096 cpr. - **Sinal:** ABN em nível TTL, *rise/fall* típicos 10 ns, $V_{OH} \geq 2.4$ V, $V_{IL} \leq 0.4$ V, $I_{out\ max} \approx 20$ mA. - **Alimentação:** 4.5–5.5 V (típico 5 V), consumo ≈ 110 mA. - **Faixa de frequência:** até ~ 1.5 MHz de comutação. - **Mecânica:** diâmetro do furo 5 mm ou 6.35 mm; carga axial 50 N, radial 80 N; rotação máxima 6000 rpm; módulo 28 mm $\times$ 28 mm $\times$ 18 mm (aprox.).

Integração com o firmware: conectar A/B aos temporizadores do STM32 em modo *encoder* (por exemplo, TIM2/TIM3/TIM5) e o *index* N a uma entrada de reset/captura para referência de zero. Converter contagens em ângulo via $\theta = 360^\circ \cdot \text{count} / \text{CPR}$. Neste TCC foi utilizada a variante **TMCS-28-10k** (código TMCS-28-x-10000-AT-01), com **625 lpr** e **40 000 cpr**; portanto, considerar CPR = 40 000. Como as saídas são TTL a 5 V, prever adaptação de nível para GPIOs de 3.3 V do STM32 (divisores/*level shifter*).

2.10.3 Implicções de projeto

No contexto deste trabalho, os recursos de microstepping e os modos *stealthChop2*/*spreadCycle* são os mais relevantes para compatibilizar a taxa de passos do DDA (50 kHz) com suavidade e torque. Quando aplicável, o uso de *StallGuard2* permite procedimentos de homing sem sensores, desde que a calibração de SGTHRS/TCOOLTHRS seja validada em bancada. A parametrização via SPI/UART se integra naturalmente à camada de *Services*, mantendo os ajustes desacoplados do laço de tempo real.

2.11 Motores de Passo NEMA 23, Passo de 0,9° e Seleção da Corrente I_{RMS}

2.11.1 NEMA 23: geometria e implicações mecânicas

NEMA 23 define **dimensões de flange** do motor (aprox. 57 mm × 57 mm) e furação de montagem; **não** define torque, corrente ou passo elétrico. Assim, motores NEMA 23 podem variar em comprimento do corpo, inércia do rotor, resistência/indutância por fase e torque de retenção. No projeto, os três eixos utilizam motores NEMA 23 por combinarem: (i) facilidade de fixação em estruturas CNC, (ii) bom compromisso entre torque e tamanho, (iii) ampla disponibilidade de *drivers* compatíveis (p.ex., TMC5160).

2.11.2 Passo elétrico: 1,8° vs 0,9°

O **ângulo de passo elétrico** é a rotação do eixo a cada *passo inteiro* sem microstepping. Os valores típicos são:

$$\theta_{\text{passo}} \in \{1,8^\circ, 0,9^\circ\} \Rightarrow N_{\text{passos/rev}} = \frac{360^\circ}{\theta_{\text{passo}}}$$

Assim, motores de 1,8° têm $N = 200$ passos/rev; motores de 0,9° têm $N = 400$ passos/rev.

Efeitos práticos do passo de 0,9°.

- **Mais resolução mecânica** por volta (400 vs 200 passos), favorecendo posicionamento fino e menor *ripple* posicional a baixas velocidades.
- **Maior exigência de taxa de pulsos** para a mesma velocidade linear, pois dobra-se o número de passos por volta. Para uma razão de microstepping M , a relação é:

$$\text{RPS} = \frac{f_{\text{STEP}}}{N \cdot M} \Rightarrow f_{\text{STEP}} = \text{RPS} \cdot N \cdot M$$

Ex.: a mesma RPS em 0,9° requer o dobro de f_{STEP} em relação a 1,8°.

- **Desempenho em alta rotação:** por demandar maior frequência elétrica, a queda de torque em alta velocidade pode ser mais perceptível em 0,9° se a eletrônica não compensar a indutância.

2.11.3 Microstepping, suavidade e taxa de passos

O **microstepping** (p.ex., $M = 16$, $M = 256$) interpola correntes senoidais nas fases, reduz ruídos e vibrações e melhora a suavidade do movimento. Contudo:

- A **resolução de comando** é limitada por f_{STEP} disponível do controle (DDA/gerador de trajetórias).
- A **capacidade de torque incremental** por micropasso é menor que o torque de passo inteiro; por isso, em carga dinâmica, é comum trabalhar com correntes moderadas e acelerações respeitando a curva torque-velocidade.

2.11.4 Parâmetros elétricos e modelo de primeira ordem

Motores de passo bipolares são especificados por I_{rated} (corrente por fase), resistência R e indutância L . A dinâmica de corrente (sem FEM de rotação) obedece:

$$\frac{di}{dt} \approx \frac{V_{bus} - v_{chopper}}{L} \quad \text{com} \quad \tau = \frac{L}{R}$$

Um *driver* de modo corrente (p.ex., TMC5160) aplica **chopping** para manter I_{RMS} alvo nas fases, usando tensão de barramento elevada (24 V a 48 V) para acelerar a resposta de corrente contra a indutância. O **aquecimento** escala como $P \propto I_{RMS}^2 R$; por isso, trabalhar com frações moderadas de I_{rated} reduz significativamente a dissipação, sem inviabilizar o torque exigido.

2.11.5 Torques relevantes

- **Torque de retenção** T_{hold} : torque estático máximo com corrente nominal e eixo travado.
- **Torque de detente** T_{det} : ondulação mecânica intrínseca (sem corrente); corrente precisa superá-lo para iniciar movimento confiável.
- **Torque dinâmico**: decresce com velocidade elétrica pela limitação de di/dt e FEM de rotação; tensão maior no barramento ajuda a preservar torque em regime.

2.11.6 Resumo para os motores usados

- **JK57HM76-2804** (NEMA 23, $I_{rated} \approx 2,8$ A, $T_{hold} \approx 1,8$ N · m): alto torque estático; adequado a eixos que demandam maior força de corte; pode ser 0,9° ou 1,8° conforme variante.
- **23HM8430 0,9°** (NEMA 23, $I_{rated} \approx 3,0$ A, $T_{hold} \approx 1,5$ N · m): passo fino nativo (400 passos/rev) favorece resolução e suavidade a baixas velocidades, exigindo maior f_{STEP} para a mesma rotação.

2.11.7 Fundamentação teórica para a seleção de I_{RMS}

Motores de passo operam com torque aproximadamente proporcional à corrente elétrica por fase, de forma que a relação entre *torque de retenção* (T_{hold}) e *corrente nominal* (I_{rated}) pode ser usada como boa aproximação da constante de torque (k_T). Assim, variações em I_{RMS} impactam diretamente na capacidade de aceleração, velocidade máxima utilizável e estabilidade mecânica da máquina CNC.

$$k_T \approx \frac{T_{hold}}{I_{rated}} \quad [\text{N} \cdot \text{m/A}]$$

Entretanto, a operação contínua em valores próximos ao limite nominal aumenta significativamente o aquecimento tanto do estator quanto do driver. Neste trabalho, considerando o uso do driver Trinamic TMC5160 a 48 V, optou-se por definir um **perfil de corrente reduzida** que garantisse torque suficiente para movimentação segura, mas mantendo temperaturas moderadas sem necessidade de refrigeração excessiva.

Para garantir movimento sem carga, é necessário superar o *torque de detente* do motor (T_{det}), valor mecânico inerente à geometria do rotor. Assim, pode-se estimar a **corrente mínima prática** como:

$$I_{\min} \approx \alpha \cdot \frac{T_{det}}{k_T}$$

onde α é um fator de segurança entre 1,5 e 2,5, para compensar atrito e irregularidades dinâmicas.

Os dois modelos selecionados (um por eixo), ambos NEMA 23, são:

- JK57HM76-2804: 2,8 A/fase, 0,9° ou 1,8°, $T_{hold} \approx 1,8 \text{ N} \cdot \text{m}$.
- 23HM8430: 3,0 A/fase, 0,9°, $T_{hold} \approx 1,5 \text{ N} \cdot \text{m}$.

Assumindo $T_{det} \approx 0,06 \text{ N} \cdot \text{m}$ (valor típico para NEMA 23), obtiveram-se os parâmetros da Tabela 2.9.

Tabela 2.9: Estimativa da corrente mínima prática (I_{RMS}).

Motor	I_{rated} [A]	T_{hold} [N·m]	k_T [N·m/A]	I_{det} [A]	I_{min} [A]
JK57HM76-2804	2,8	1,8	0,643	0,093	0,14–0,23
23HM8430 0,9°	3,0	1,5	0,50	0,12	0,18–0,30

Do ponto de vista térmico e de robustez, adotou-se no firmware:

- JK57HM76-2804: **$I_{RUN} \approx 0,28 \text{ A}$** e **$I_{HOLD} \approx 0,12 \text{ A}$** .
- 23HM8430: **$I_{RUN} \approx 0,30 \text{ A}$** e **$I_{HOLD} \approx (0,12\text{--}0,15) \text{ A}$** .

Esses valores representam apenas **8–12% da corrente nominal**, o que reduz expressivamente a potência dissipada ($P \propto I_{RMS}^2 R$), sem prejuízo significativo sobre o torque necessário para os movimentos previstos no CNC desenvolvido. Caso o processo exija maior aceleração ou usinagem mais agressiva, o ajuste de I_{RUN} pode ser realizado dinamicamente pelo firmware via registro `IHOLD_IRUN` do TMC5160.

Assim, a fundamentação e a calibração experimental convergiram para uma configuração segura, eficiente e com excelente relação entre torque ofertado e controle térmico do sistema eletromecânico.

2.12 Registros de Configuração Utilizados

Esta seção sumariza os principais registros configurados no STM32L475 e no driver TMC5160 conforme empregados neste trabalho. O formato segue o exemplo de tabelas de configuração com campos e finalidade.

2.12.1 STM32L475

Tabela 2.10: Registros STM32 L4 utilizados neste trabalho (ver [STMicroelectronics, 2013, STMicroelectronics, 2018a]).

Perif.	Registro	Campo/Valor	Finalidade
TIM6	PSC	79	Prescaler para 50 kHz (DDA).
TIM6	ARR	19	Período de 20 μ s.
TIM7	PSC	7999	Prescaler para 1 kHz (PID).
TIM7	ARR	9	Período de 1 ms.
TIM2/3/5	SMCR	SMS=0b011	Modo encoder (contagem em TI1 e TI2).
TIM2/3/5	CCMR1	CC1S=01; CC2S=01	Entradas mapeadas para TI1/TI2.
TIM2/3/5	CCER	CC1P=0; CC2P=0	Polaridade não invertida (conforme ligação).
SPI1	CR1	MSTR=0; CPOL=0; CPHA=0	Modo escravo, fase/polaridade padrão.
SPI1	CR2	RXDMAEN=1; TXDMAEN=1; DS=8	SPI com DMA e palavra de 8 bits.
DMAx	CCR(RX/TX)	MINC=1; CIRC=1; DIR	DMA circular para SPI1 RX/TX.
DMAx	CNDTR/CPAR/CMAR	–	Tamanho e endereços de perif. e memória.
USART1	BRR	115200	Baud rate para logs/telemetria.
EXTI	IMR/RTSR/FTSR	Linhas de E-STOP/limites	Interrupções de segurança.
NVIC	PRI0	–	Priorização: segurança, TIM6, SPI/DMA, TIM7, USART.

2.12.2 TMC5160

Tabela 2.11: Registros TMC5160 utilizados neste trabalho (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]).

Registro	Campo/Valor	Finalidade
IHOLD_IRUN	IHOLD, IRUN, IHOLDDELAY	Correntes de hold/run e rampa de corrente.
TPOWERDOWN	Tempo (μ s)	Tempo para redução de corrente em inatividade.
CHOPCONF	spreadCycle/ hysteresis	Chopper de corrente e modo de comutação.
PWMCONF	stealthChop2 (PWM_AMPL/GRAD)	Chaveamento silencioso e parâmetros PWM.
SGTHRS	Limiar	Limiar do StallGuard2 para homing <i>sensorless</i> .
TCOOLTHRS	Velocidade limiar	Janela de atuação para StallGuard/coolStep.
TPWMTHRS	Velocidade limiar	Limite de comutação stealth-Chop2 $\leftrightarrow$ spreadCycle.
DRV_STATUS	Flags	Diagnóstico: sobrecorrente, subtensão, temperatura, SG_RESULT.

Observação: valores exatos de bits podem variar conforme a placa/variante e roteiro de testes; recomenda-se validar com os manuais do STM32 L4 ([STMicroelectronics, 2018a]) e o datasheet do TMC5160 ([TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]).

Capítulo 3

Metodologia

A metodologia adotada para o desenvolvimento do controlador CNC seguiu um roteiro incremental que prioriza a validação dos blocos críticos antes de incorporar funcionalidades auxiliares. Cada etapa foi documentada, testada e revisada de forma a garantir que os requisitos de determinismo fossem mantidos durante todo o processo.

3.1 Roteiro incremental de inicialização

O ponto de partida consistiu na configuração do clock principal para 80 MHz e na definição das prioridades do NVIC de acordo com o orçamento temporal: interrupções externas de segurança, TIM6, SPI1/DMA, TIM7 e USART1. Em seguida, foram ativadas as entradas de parada de emergência (E-STOP) e sensores de proximidade, assegurando que flags de segurança pudessem interromper o fluxo de comandos. As etapas posteriores focaram na calibração dos temporizadores: o TIM6 foi dimensionado com $PSC = 79$ e $ARR = 19$, enquanto o TIM7 recebeu $PSC = 7999$ e $ARR = 9$. Os temporizadores TIM2, TIM3 e TIM5 foram configurados em modo quadratura para leitura de encoders.

A configuração do SPI1 em modo escravo com DMA circular foi realizada após os temporizadores, evitando contenda na memória compartilhada. Por fim, a USART1 foi ajustada para 115 200 bps e integrada a um serviço de log com fila não bloqueante. Cada etapa do roteiro foi acompanhada por testes de bancada: medições de frequência com osciloscópio, leitura de contadores de encoder e injeção de quadros SPI utilizando o cliente Python.

3.2 Arquitetura de software

O firmware foi estruturado em três camadas principais. A camada *Core* engloba os artefatos gerados pelo STM32CubeMX, incluindo inicialização de periféricos, descrições de pinos e funções HAL. Sobre ela, a camada *App* implementa o laço principal (`app_poll`), o agendador de serviços e as rotinas de inicialização específicas do projeto. A camada *Services* agrupa módulos especializados, como o gerador de passos, o controlador PID, o serviço de homing e o roteador de mensagens SPI.

A fila de recepção SPI é mantida em memória circular, preenchida pelas rotinas de interrupção e consumida por `app_poll`. Respostas são enfileiradas em `g_app_responses` e promovidas para o buffer de DMA quando disponíveis. A integração com os drivers TMC5160 ocorre por meio de uma API dedicada que abstrai comandos STEP/DIR/EN e monitora condições de falha.

3.2.1 Fluxo do comando `cnc-cli`

O comando `cnc-cli` executa uma sequência declarativa de ações em um arquivo JSON (patches do TMC5160, ajustes de PID e movimentos). A Figura ?? resume o fluxo principal, incluindo validações, enfileiramento no STM32 e monitoramento de falhas do TMC5160.

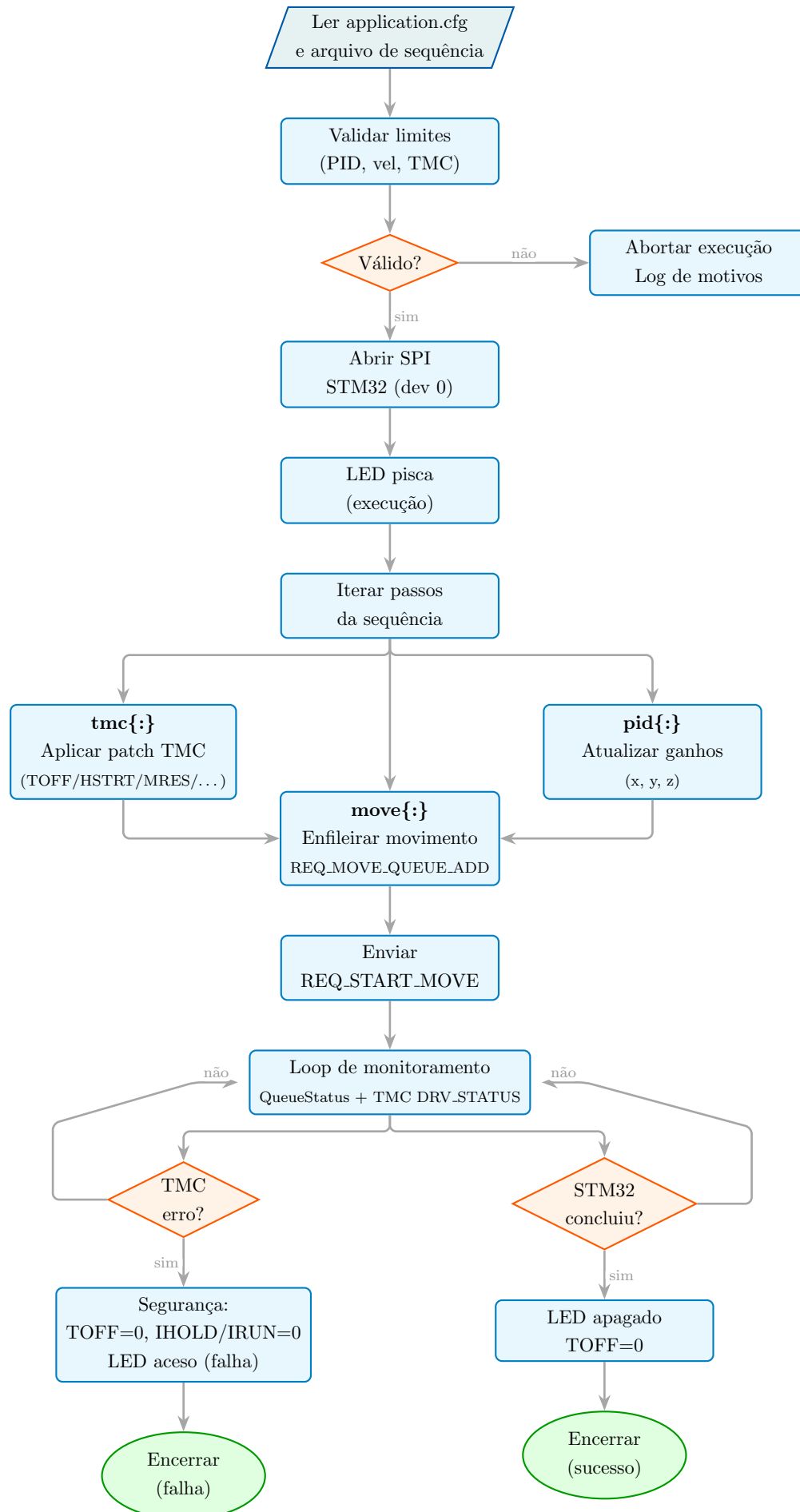


Figura 3.1: Fluxo do comando `cnc-cli`: validação, aplicação de patches TMC, enfileiramento de movimentos no STM32, monitoramento e estados do LED.

Descrição do fluxo

1. **Leitura de arquivos:** o cliente carrega o `application.cfg` (conexões, limites e presets) e o JSON de sequência (lista de passos `steps`).
2. **Validação** (no Raspberry): cada item da sequência é verificado contra os limites do `cfg.limits`—faixas de PID, teto de velocidade e parâmetros do TMC5160. Qualquer violação aborta a execução, com mensagens registradas em log (`raspberrypi/run_logs/`).
3. **Conexão SPI com STM32:** abre `/dev/spidev{bus}.{dev}` do microcontrolador para enviar comandos do protocolo CNC (`MOVE_QUEUE_ADD/STATUS/START/END`).
4. **Sinalização por LED (STM32):** o LED passa a piscar durante a execução; em falha fica aceso; no sucesso, é apagado ao final.
5. **Iteração de passos:**
 - `tmc{...}`: aplica patches no `CHOPCONF/PWMCONF`/registradores correlatos (`toff`, `hstrt/hend`, `tbl`, `dedge`, `microsteps/interpolate`, `globalscaler`, `ihold/irun/ihold_delay`, `tpowerdown`).
 - `pid{...}`: atualiza os ganhos padrão por eixo para os próximos movimentos.
 - `move{pos, vel, dir}`: enfileira segmentos no STM32 via `REQ_MOVE_QUEUE_ADD` com `frameId` gerado internamente e, após o último, envia `REQ_START_MOVE`.
6. **Monitoramento:** o cliente consulta periodicamente o `MOVE_QUEUE_STATUS` para acompanhar o progresso (percentuais por eixo). Em paralelo, lê `GSTAT/DRV_STATUS` do TMC5160 para detectar falhas críticas.
7. **Tratamento de falhas (TMC):** se houver `overtemp/OTPW` ou `driver_error`, aplica “segurança” em todos os drivers (`TOFF=0`, `IHOLD/IRUN=0`), mantém o LED do STM32 aceso e encerra a execução.
8. **Término normal:** ao concluir todos os movimentos, o cliente aplica `TOFF=0` (pós-execução), apaga o LED do STM32 e finaliza o run.

3.3 Procedimentos de teste

Os testes de validação foram conduzidos em três frentes. Primeiro, medições com osciloscópio e analisador lógico verificaram a frequência dos pulsos STEP e a variação temporal (*jitter*) do TIM6, confirmando a estabilidade em 50 kHz. Em seguida, foram realizadas varreduras de ganho nos controladores PID para avaliar margem de fase e resposta a degraus, utilizando logs exportados via USART1. Por fim, o pipeline SPI foi monitorado com o cliente `cnc_spi_client.py`, observando a necessidade de até três ciclos de enquete para respostas completas e avaliando o impacto de diferentes valores de `APP_SPI_RESTART_DEFER_MAX`. Os dados coletados subsidiam as análises apresentadas no Capítulo 4.

3.4 Identificação experimental do modelo FOPDT

Esta seção descreve o procedimento de identificação adotado para estimativa de K , L e τ a partir de respostas a degrau de velocidade obtidas em bancada. Os ensaios foram conduzidos com o firmware instrumentado para emitir amostras de posição e velocidade via USART e SWV, em formato CSV, a partir das rotinas de telemetria do `motion_service.c`. Para cada combinação de eixo e microstepping, foi aplicado um perfil de degrau de velocidade gerado pelo DDA, e os Sinais de referência (`v_cmd.sps`)

e de medição (velocidade do encoder) foram registrados para posterior processamento em Python. O método opera inteiramente no domínio do tempo, em linha com as abordagens clássicas de identificação por curva de reação descritas por [Åström and Hägglund, 1995], e mostra-se robusto a níveis moderados de ruído.

3.4.1 Pré-processamento e séries derivadas

Considere amostras ordenadas por tempo contendo os campos (t, p, c) , em que p representa a contagem acumulada de pulsos do atuador (STEP) e c a contagem acumulada do encoder (contagens). A partir de pares ou de uma janela deslizante de comprimento N , definem-se

$$\Delta t_k = t_k - t_{k-N+1}, \quad \Delta p_k = p_k - p_{k-N+1}, \quad \Delta c_k = c_k - c_{k-N+1}, \quad (3.1)$$

$$v_{\text{cmd}}(k) = \frac{\Delta p_k}{\Delta t_k}, \quad v_{\text{enc}}(k) = \frac{\Delta c_k}{\Delta t_k}. \quad (3.2)$$

Para reduzir o efeito de ruído, utiliza-se tipicamente $N \in [5, 10]$. Pares com $\Delta t_k \leq 0$ são descartados. O valor de regime (*steady state*) de cada série é a média dos últimos $\rho \in (0, 1)$ da janela temporal disponível (*e.g.*, $\rho = 0,2$), denotados por $\bar{v}_{\text{cmd}}$ e $\bar{v}_{\text{enc}}$.

Nesta notação, $f(k)$ indica o valor da grandeza discreta f na k -ésima amostra, usualmente associado ao instante $t = kT_s$. Em particular, $v_{\text{cmd}}(k)$ representa a velocidade de comando (pulsos/s) derivada dos pulsos emitidos, enquanto V_{cmd} será utilizado para denotar a amplitude do degrau de comando nas simulações e validações (definido como $V_{\text{cmd}} := \bar{v}_{\text{cmd}}$).

3.4.2 Extração de marcos temporais

Define-se o *início efetivo* do degrau nos sinais de comando e de medição pelos primeiros cruzamentos de 5% de seus respectivos regimes:

$$t_{\text{cmd}}^{5\%} = \inf\{t : v_{\text{cmd}}(t) \geq 0,05 \bar{v}_{\text{cmd}}\}, \quad t_{\text{enc}}^{5\%} = \inf\{t : v_{\text{enc}}(t) \geq 0,05 \bar{v}_{\text{enc}}\}. \quad (3.3)$$

Analogamente, obtém-se o instante t_{63} como o primeiro cruzamento de 63,2% no sinal medido:

$$t_{63} = \inf\{t : v_{\text{enc}}(t) \geq 0,632 \bar{v}_{\text{enc}}\}. \quad (3.4)$$

3.4.3 Estimativas K , L e τ

O ganho estático é dado por $K = \bar{v}_{\text{enc}}/\bar{v}_{\text{cmd}}$. O tempo morto é $L = \max\{0, t_{\text{enc}}^{5\%} - t_{\text{cmd}}^{5\%}\}$. A constante de tempo decorre de $\tau = t_{63} - L$. Quando limitações de amostragem impõem baixa resolução temporal, adota-se o piso $\Delta t_{\min}$ observado na série crua, substituindo L e/ou τ por $\Delta t_{\min}$ quando as estimativas resultarem nulas ou negativas.

3.4.4 Síntese PD e validação

Com as estimativas (K, L, τ) , computam-se os ganhos K_p e K_d por meio das regras de Ziegler–Nichols para curva de reação (com $K_i = 0$ na malha de velocidade). A validação consiste em aplicar as constantes ao modelo FOPDT discretizado ($T_s = 1$ ms) e comparar a resposta simulada à resposta medida, verificando sobre-elevação, tempo de subida e ausência de saturações relevantes.

3.4.5 Validação gráfica do modelo FOPDT

Para documentar a aderência do modelo FOPDT aos dados, foram gerados gráficos de sobreposição entre a velocidade medida (encoder) e a resposta teórica do FOPDT a um degrau. A resposta empregada decorre da solução analítica do sistema de 1ª ordem com atraso sob degrau atrasado (ver Seção 2.6.1): $y(t) = KU[1 - e^{-(t-L)/\tau}]$ para $t \geq L$ e 0 caso contrário. A amplitude U é tomada como a velocidade de comando em regime, $U := \bar{v}_{\text{cmd}}$, garantindo que o patamar previsto (KU) coincida com o observado em regime quando a aproximação é válida. As figuras a seguir apresentam, para cada eixo, os três microsteppings avaliados.

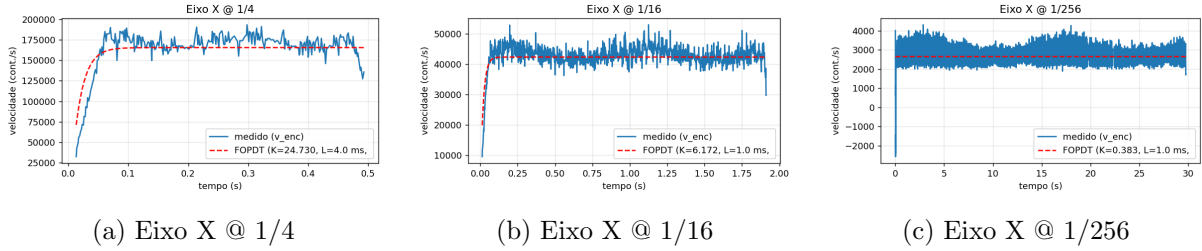


Figura 3.2: Sobreposição entre velocidade medida e FOPDT (eixo X).

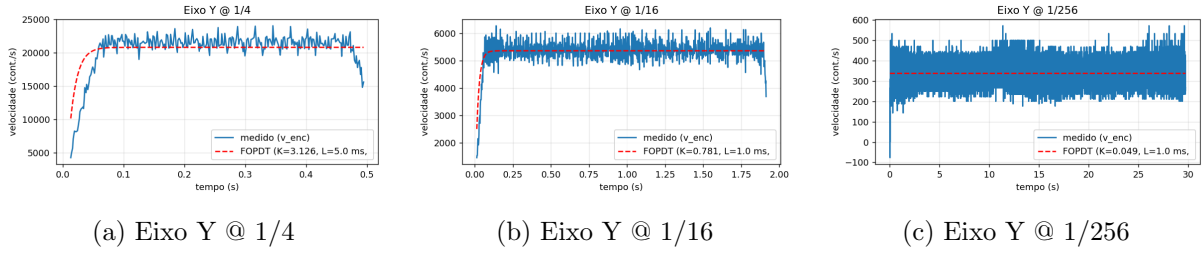


Figura 3.3: Sobreposição entre velocidade medida e FOPDT (eixo Y).

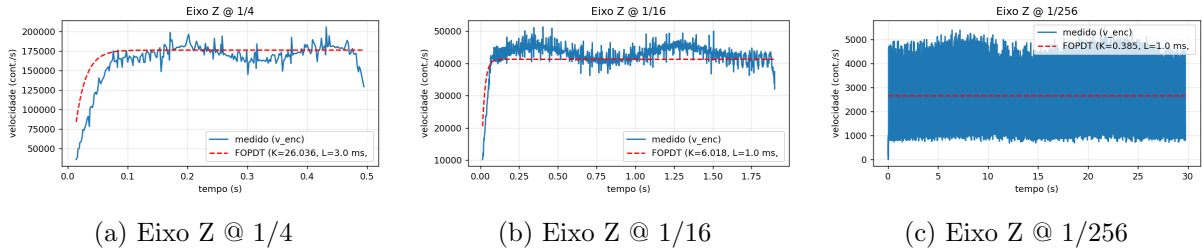


Figura 3.4: Sobreposição entre velocidade medida e FOPDT (eixo Z).

Observa-se, para microsteppings altos (1/256), tempos característicos menores ($\tau \approx 1$ ms) e ganhos estáticos reduzidos, ao passo que microsteppings baixos (1/4) elevam o ganho efetivo e alongam L ou τ conforme o eixo. As discrepâncias residuais decorrem de ruído de medição, quantização e não linearidades (*e.g.*, atrito viscoso e limitações de resolução temporal), porém a forma global é adequadamente capturada pelo FOPDT.

3.5 Arquitetura de Hardware

Esta seção descreve o circuito do controlador, os blocos físicos e as conexões empregadas entre a placa STM32, os drivers de potência TMC5160 e o encoder óptico incremental TMCS-28. Também são listadas

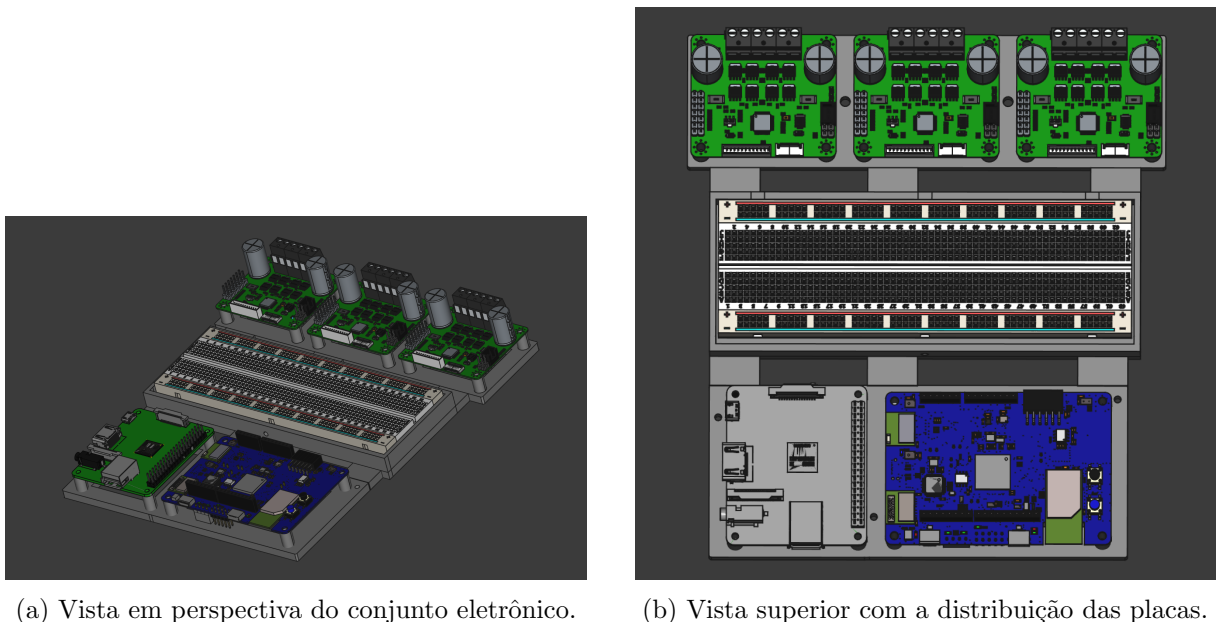
recomendações de montagem, alimentação e EMC.

3.5.1 Visão geral do sistema

O sistema é composto por:

- **Lógica:** placa com STM32L475 operando a 3.3V, cristal de referência e interfaces SPI/USART/GPIO.
- **Drivers de eixo:** módulos/placa com **TMC5160** recebendo sinais STEP/DIR/EN e configurados via SPI.
- **Realimentação:** encoder óptico **TMCS-28** (ABN, TTL 5V) acoplado ao eixo principal.
- **Alimentação:** trilha de 5V para lógica e sensores; regulador 3.3V local para o microcontrolador; alimentação separada/filtrada para os estágios de potência dos motores.

A Figura 3.5 apresenta o modelo tridimensional do conjunto do controlador CNC, destacando a placa STM32L475, o módulo de interface com a Raspberry Pi, a base de prototipagem e as placas de potência TMC5160 posicionadas na região superior. Essa organização em níveis facilita o roteamento de sinais de baixa tensão e a separação física dos estágios de potência.



(a) Vista em perspectiva do conjunto eletrônico.

(b) Vista superior com a distribuição das placas.

Figura 3.5: Modelo 3D do controlador CNC com placa STM32, interface para Raspberry Pi, base de prototipagem e módulos de potência TMC5160.

3.5.2 Alimentação e EMC

- **5V lógica:** entrada regulada (fonte externa), com capacitores de granel (10 μ F a 47 μ F) e desacoplamentos locais (100 nF) próximos aos CI.
- **3.3V:** regulador LDO dedicado ao STM32 e periféricos 3.3V. Seguir recomendações de estabilidade do LDO (ESR dos capacitores) e planos de terra.
- **Aterramento:** retorno de alta corrente dos motores mantido separado do plano de lógica; conexão em estrela/próximo ao ponto de entrada da energia. Loops curtos para sinais comutação.

3.5.3 Drivers TMC5160

- **Sinais:** STEP/DIR/EN do STM32 (3.3V) para TMC5160. Interposição de *level shifter* se necessário conforme a placa utilizada.
- **SPI:** barramento dedicado para configuração (SCK, MOSI, MISO, CS_x). Resistores de terminação/ série curtos para integridade de sinal.
- **Potência:** desacoplamento de VM

com capacitores de baixa ESR; rotação larga para trilhas de corrente. - **Proteções:** leitura de `DRV_STATUS` para falhas e `EN` global intertravado pelo `E-STOP`.

3.5.4 Encoder óptico TMCS-28

O encoder óptico incremental TMCS-28 fornece três saídas digitais em nível TTL 5 V: canais **A** e **B** em quadratura e canal **N** (*index*). Para compatibilização com o STM32, todos os sinais são condicionados para 3.3 V por meio de divisores resistivos, garantindo margens de ruído adequadas.

No microcontrolador, os canais A/B são ligados a periféricos com modo *encoder* habilitado: o eixo X utiliza o **TIM3** (16 bits), o eixo Y utiliza o **LPTIM1** (16 bits em modo encoder) e o eixo Z utiliza o **TIM5** (32 bits), conforme o mapeamento definido em `motion_hw.c`. Nesses modos, o caminho entre as entradas A/B e o registrador de contagem é implementado em hardware: a cada transição em quadratura o contador interno é atualizado diretamente, sem que a CPU precise tratar interrupções ou movimentar dados via DMA, o que reduz sobrecarga e variações temporais na medição de posição.

O canal N é conectado a uma entrada de captura/interrupção e utilizado como referência de zero: durante a rotina de homing, um pulso em N permite associar um “zero absoluto” à posição mecânica do ponteiro.

Neste trabalho emprega-se o modelo **TMCS-28-10k**, com resolução efetiva de `ENC_COUNTS_PER_REV = 40 000` contagens por volta do ponteiro (considerando a relação mecânica do conjunto e a decodificação em quadratura). A conversão de contagens para ângulo segue

$$\theta [^\circ] = 360^\circ \cdot \frac{\text{count}}{\text{ENC_COUNTS_PER_REV}},$$

de modo que, para os eixos X/Z, utiliza-se `ENC_COUNTS_PER_REV = 40 000`. A velocidade em rotações por minuto (rpm) é obtida a partir da taxa de contagens por segundo (CPS):

$$\text{rpm} = 60 \cdot \frac{\text{CPS}}{\text{ENC_COUNTS_PER_REV}}.$$

A Figura 3.6 apresenta o modelo em CAD do suporte impresso em 3D projetado para acoplar o encoder TMCS-28 ao eixo do motor de passo e fornecer um ponteiro visível. O suporte garante alinhamento axial, espaço para o disco graduado e pontos de fixação compatíveis com o padrão NEMA 23. A Figura 3.7 mostra a implementação física, com o encoder e o ponteiro instalados nos três eixos do protótipo.

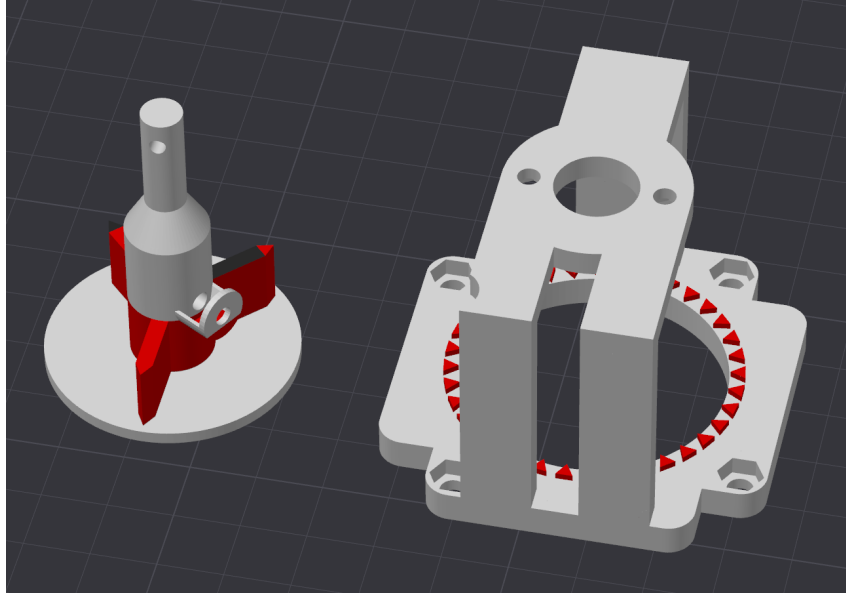


Figura 3.6: Modelo 3D do suporte do encoder TMCS-28 e do ponteiro acoplado ao eixo do motor de passo.

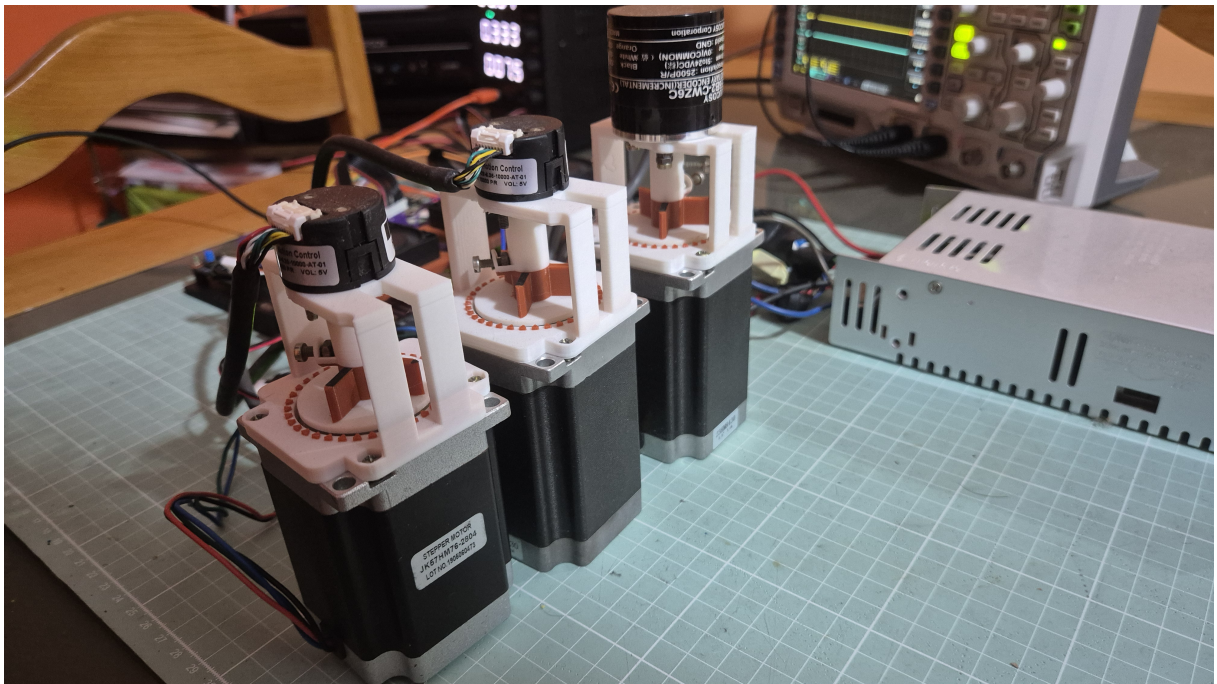


Figura 3.7: Vista dos motores de passo com suportes impressos em 3D, ponteiros e encoder TMCS-28 acoplado ao eixo para realimentação de posição.

3.5.5 Intertravamentos e segurança

A proteção elétrica dos motores de passo baseados no TMC5160 faz uso dos avisos de tensão e corrente expostos nos registradores `GSTAT` e `DRV_STATUS` do driver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]. Esses registradores indicam condições como subtensão, sobretensão/curto nas fases e sobretensão (*OT/OTPW*). No sistema desenvolvido, tais flags são lidos periodicamente pelo cliente na Raspberry Pi (`cnc_cli`), servindo como

diagnóstico indireto do estado dos motores e do cabeamento: a ausência de falhas sugere operação normal, enquanto qualquer alerta indica degradação ou condição de falha.

Quando uma falha crítica é detectada (por exemplo, *overtemp*, *OTPW* ou *driver_error*), aplica-se uma rotina de segurança que desliga o estágio de potência de forma controlada: o script força `IHOLD = IRUN = 0`, ajusta `CHOPCONF.TOFF = 0` e configura `PWMCONF.FREEWHEEL = 1`, removendo corrente das bobinas e colocando o TMC5160 em estado seguro. Esse mecanismo complementa as proteções internas do driver e fornece um meio simples de interromper o sistema sempre que os avisos de tensão ou temperatura indicarem condições anormais.

3.5.6 PCB, cabeamento e montagem

- **Layout:** a base mecânica foi organizada em três níveis bem definidos. A região inferior abriga a placa STM32 e a Raspberry Pi, com trilhas de lógica e alimentação de baixa potência. Sobre elas, uma área central de *protoboard* foi reservada exclusivamente para acomodação de conexões, divisores e pequenos ajustes de circuito, facilitando reconfigurações sem retrabalho de PCB. Na parte superior ficam os módulos TMC5160 e os dissipadores, mantendo o domínio de potência fisicamente afastado do microcontrolador e da Raspberry Pi.

- **Cabeamento:** pares A/B do encoder e sinais STEP/DIR são roteados em pares trancados, reduzindo a suscetibilidade a ruído eletromagnético. As linhas de SPI entre STM32, TMC5160 e Raspberry Pi são mantidas curtas e com retorno próximo, minimizando laços de corrente.

- **Ordem de testes:** a comissionação do hardware segue uma sequência conservadora: (i) validação da alimentação e dos reguladores; (ii) verificação de clock e USART; (iii) testes do SPI com os TMC5160; (iv) leitura incremental ABN do TMCS-28; e, somente ao fim, (v) acionamento dos motores sem carga, antes de aplicar perfis mais agressivos de corrente e velocidade.

A Figura 3.8 mostra a montagem do protótipo em bancada, com a fonte de alimentação, os três motores de passo, o conjunto de drivers TMC5160, a placa STM32 e a Raspberry Pi interligados. A disposição segue as recomendações de separação entre domínios de potência e lógica descritas nesta seção.

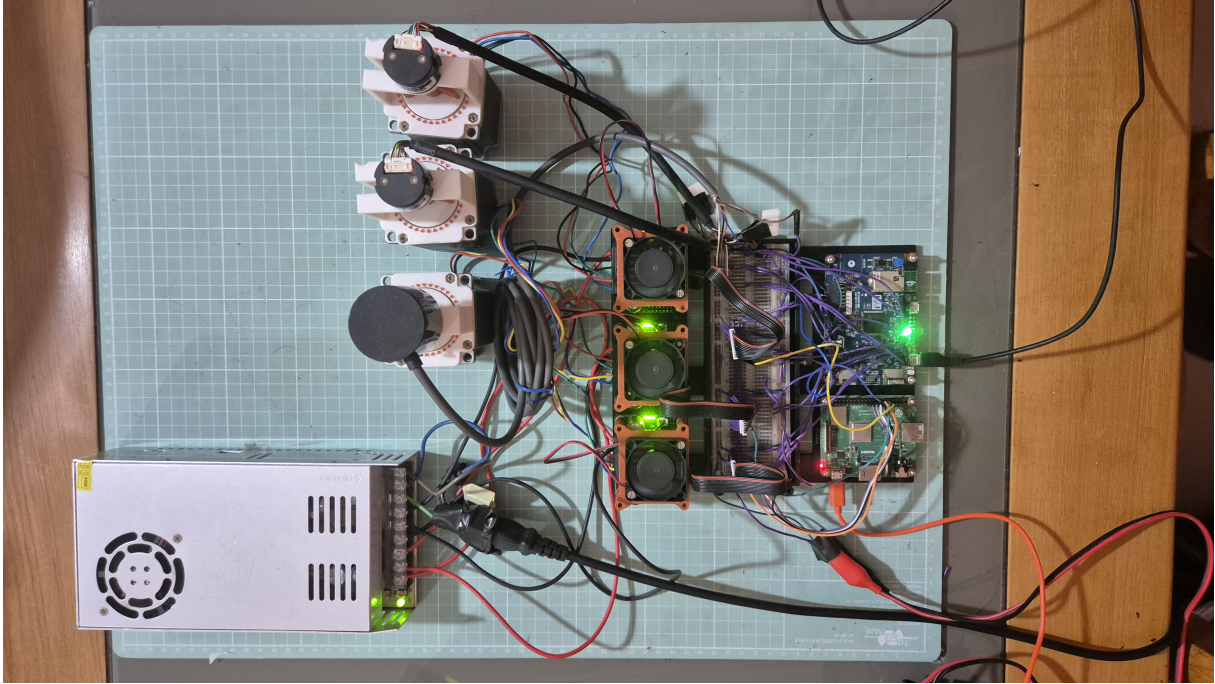


Figura 3.8: Protótipo do controlador CNC montado em bancada, com fonte de alimentação, motores de passo, drivers TMC5160, placa STM32 e Raspberry Pi.

Como referência adicional de montagem e integração de hardware, ver [Romeros, 2022].

3.6 Arquitetura de Controle de Movimento (Visão Geral)

O subsistema de movimento foi dividido em dois laços temporizados: (i) **TIM6** a 50 kHz (20 μ s por tick), dedicado à geração de pulsos *STEP* a partir de um DDA (Digital Differential Analyzer) em ponto fixo; e (ii) **TIM7** a 1 kHz (1 ms), dedicado à lógica de rampa trapezoidal (aceleração/desaceleração), ajuste de velocidade efetiva, leitura dos encoders e controle PI de posição. A separação reduz variações temporais na largura do *STEP* e garante que o cálculo das rampas e do PI não impacte o *timing* do pulso. As constantes de tempo usadas no código são: `MOTION_TIM6_HZ=50000`, `MOTION_STEP_HIGH_TICKS=1`, `MOTION_STEP_LOW_TICKS=1`, `MOTION_DIR_SETUP_TICKS=1`, `MOTION_ENABLE_SETTLE_TICKS=2`.

3.6.1 Compatibilidade de *timing* com o TMC5160 (*STEP/DIR*)

O TMC5160 possui um pequeno filtro analógico nas entradas digitais ($t_{\text{FILTS}} \approx 20 \text{ ns}$) para rejeitar ruídos estreitos. Por isso, o datasheet especifica que os tempos em nível alto (t_{SH}) e em nível baixo (t_{SL}) do sinal *STEP* devem obedecer $t_{\text{SH}}, t_{\text{SL}} \geq \max\{t_{\text{FILTS}}, t_{\text{CLK}} + 20 \text{ ns}\}$, resultado em pulsos efetivos tipicamente acima de 100 ns. Além disso, o relacionamento entre *DIR* e *STEP* deve respeitar tempos de *setup/hold*: pelo menos 20 ns entre uma variação em *DIR* e o próximo pulso de *STEP* (t_{DSU}), e mais 20 ns após o pulso antes de alterar novamente *DIR* (t_{DSH}) [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),]. Em termos práticos, isso garante que o driver tenha tempo suficiente para amostrar o sentido (*DIR*) e reconhecer cada pulso de passo (*STEP*) sem incertezas.

No projeto deste TCC, o gerador de pulsos opera com o **TIM6** a 50 kHz, o que significa um *tick* fixo de 20 μ s. A partir dele foram definidos tempos de guarda muito maiores que os limites do datasheet, introduzindo uma margem de segurança robusta:

- Largura alta de STEP: $t_{SH} = \text{MOTION_STEP_HIGH_TICKS} \cdot 20 \mu\text{s} = 20 \mu\text{s}$;
- Tempo baixo entre pulsos: $t_{SL} = \text{MOTION_STEP_LOW_TICKS} \cdot 20 \mu\text{s} = 20 \mu\text{s}$;
- *Setup* de DIR: $t_{DSU} = \text{MOTION_DIR_SETUP_TICKS} \cdot 20 \mu\text{s} = 20 \mu\text{s}$;
- *Settle* de ENABLE: $\text{MOTION_ENABLE_SETTLE_TICKS} \cdot 20 \mu\text{s} = 40 \mu\text{s}$.

Comparação do Pulso de STEP: Firmware vs. Datasheet

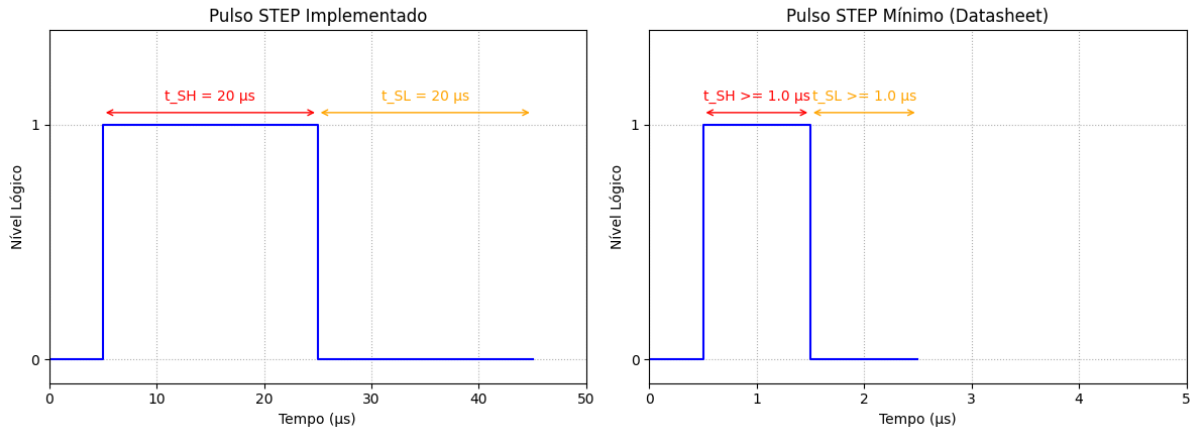


Figura 3.9: Comparação visual entre o pulso de STEP implementado no firmware (esquerda) e o pulso mínimo requerido pelo datasheet do TMC5160 (direita), evidenciando a robusta margem de tempo utilizada.

Logo, todos os requisitos do TMC5160 são amplamente atendidos com folga (vide Tabela 3.1).

Tabela 3.1: Comparativo entre tempos mínimos do TMC5160 e tempos configurados no firmware.

Parâmetro	TMC5160 (mín.)	Projeto	Margem aproximada
t_{SH} (alto de STEP)	$\gtrsim 100 \text{ ns}$	20 μs	$\times 200,000$
t_{SL} (baixo de STEP)	$\gtrsim 100 \text{ ns}$	20 μs	$\times 200,000$
t_{DSU} (DIR $\rightarrow$ STEP)	20 ns	20 μs	$\times 1,000$
t_{DSH} (STEP $\rightarrow$ DIR)	20 ns	20 μs	$\times 1,000$
ENABLE settle	n. d.	40 μs	—

Com $t_{SH} = t_{SL} = 20 \mu\text{s}$, o período mínimo de STEP é 40 μs e a frequência máxima física por eixo fica:

$$\text{MOTION_MAX_SPS} = \frac{\text{MOTION_TIM6_HZ}}{\text{MOTION_STEP_HIGH_TICKS} + \text{MOTION_MIN_LOW_TICKS}} = \frac{50\,000}{1 + 1} = 25 \text{ kSPS}.$$

3.6.2 MicroPlyer, resolução e DEDGE

Quando `intpol=1` em `CHOPCONF`, o TMC5160 interpola cada pulso de STEP até 256 microsteps (*MicroPlyer*), gerando correntes intermediárias nas bobinas mesmo que o host envie passos com resolução

mais grossa (por exemplo, 1/16). Isso aumenta a suavidade mecânica sem exigir frequências muito altas no sinal STEP.

O sinal *STEP* pode ser visto como um pulso retangular com período T e tempo em nível alto t_{alto} . A razão entre esses valores define o *duty cycle* (ou simplesmente *duty*),

$$D = \frac{t_{\text{alto}}}{T},$$

isto é, a fração do período em que o sinal permanece em nível alto. O bit **dedge** do TMC5160 controla como esse pulso é interpretado: com **dedge**=0, apenas a borda de subida conta como um passo; com **dedge**=1, tanto a borda de subida quanto a de descida passam a ser contabilizadas. Nesse segundo modo, o driver assume implicitamente um duty próximo de 50%, de forma que os passos "adiantados" e "atrasados" fiquem simétricos dentro de cada período T .

No sistema descrito neste trabalho, os pulsos de *STEP* têm largura fixa e duty bem controlado, definidos pelos parâmetros **MOTION_STEP_HIGH_TICKS** e **MOTION_STEP_LOW_TICKS**. Optou-se por manter **dedge**=0 e usar apenas a borda de subida como evento de contagem, o que simplifica o *timing*, garante estabilidade temporal, evita assimetrias desnecessárias e reduz a sensibilidade a pequenas variações de duty. Para detalhes adicionais sobre *MicroPlyer*, **intpol** e **dedge**, ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),].

3.7 DDA em Ponto Fixo (TIM6 @ 50 kHz)

A Figura 3.10 ilustra o funcionamento interno do DDA para um movimento de 7 passos em X e 4 em Y, conforme implementado no firmware. A cada tick do temporizador (eixo horizontal), os acumuladores de fase de X e Y são incrementados por valores fixos, proporcionais às suas respectivas velocidades. O eixo X, sendo mais rápido para este vetor, tem um incremento maior e seu acumulador transborda (*overflow*) mais frequentemente. Cada transbordamento, marcado pela volta do acumulador a zero no gráfico superior, corresponde à geração de um pulso STEP para o eixo, como visto no gráfico inferior. Ao final do processo, 7 pulsos foram gerados para X e 4 para Y, distribuídos no tempo de forma a manter a trajetória da reta definida pelo host.

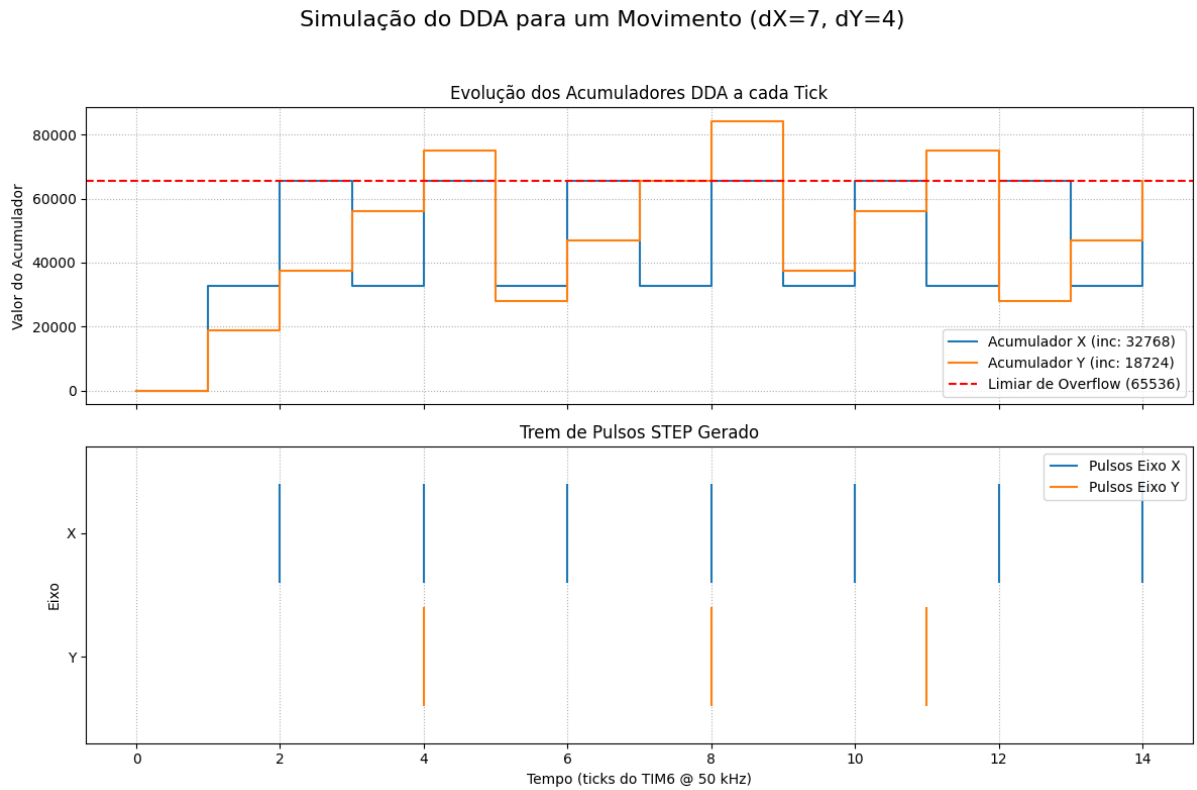


Figura 3.10: Simulação da operação interna do DDA para um movimento de 7 passos em X e 4 em Y. O gráfico superior mostra a evolução dos acumuladores de fase, e o inferior mostra os pulsos STEP gerados para cada eixo a cada transbordamento.

O gerador de passos utiliza um DDA em ponto fixo Q16.16, em que a parte inteira representa o número de passos emitidos e a parte fracionária acumula frações de passo a cada tick. Essa organização segue a formulação clássica de geradores DDA para interpoladores de movimento multi-eixo descritos na literatura de CNC [Fussell, 2003, IDC Technologies, 2014, Koren, 1978, Wang and Hu, 2016]. Em termos de fluxo, o DDA atua como um integrador digital de velocidade: a cada amostra, a referência de velocidade (em *steps/s*) é convertida em incremento de fase e, sempre que a fase acumulada cruza um limiar unitário, um novo pulso STEP é emitido. Para cada eixo i , mantemos o seguinte estado interno:

- acumulador `dda_accum_q16`, que guarda a fase acumulada em formato Q16.16;
- incremento `dda_inc_q16`, proporcional à velocidade efetiva;
- velocidade efetiva `v_actual_sps` (em *steps/s*), atualizada a cada 1 ms pelo TIM7;
- contadores de largura do STEP: `step_high` e `step_low`, usados para controlar o duty do pulso.

A cada tick do TIM6 (20 μ s), o DDA integra a velocidade em fase fracionária:

$$\text{dda_accum_q16} \leftarrow \text{dda_accum_q16} + \text{dda_inc_q16}.$$

Sempre que o acumulador ultrapassa um passo inteiro em Q16.16 ($\text{dda_accum_q16} \geq \text{Q16.1}$), considera-se que mais um passo deve ser emitido. Nessa condição, o firmware gera um pulso de *STEP*:

$$\text{step_high} \leftarrow \text{MOTION_STEP_HIGH_TICKS}, \quad \text{emitted_steps} \leftarrow \text{emitted_steps} + 1,$$

mantendo o pino em nível alto durante `step_high` e, em seguida, em nível baixo por pelo menos `step_low`

(t_{SL}). Dessa forma, cada *crossing* de Q16_1 no acumulador corresponde a um pulso STEP com largura e espaçamento bem definidos.

O incremento `dda_inc_q16` é calculado diretamente a partir da velocidade efetiva desejada:

$$\text{dda_inc_q16} = \text{Q16}\left(\frac{\text{v_actual_sps}}{\text{MOTION_TIM6_HZ}}\right).$$

Intuitivamente, isso significa que, em média, `v_actual_sps` passos são gerados a cada segundo quando o temporizador opera a `MOTION_TIM6_HZ`. Essa relação garante linearidade entre a velocidade em *steps/s* e a taxa de *crossing* do DDA, mantendo variações temporais sub- μs e duty fixo no sinal STEP (ver função `motion_on_tim6_tick`).

Algoritmo 1 Atualização do DDA Q16.16 no TIM6 (por eixo)

```

1: procedure DDA_TICK(axis)
2:                                     ▷ Executado a cada tick do TIM6 (20 µs)
3:   if step_high > 0 then
4:     step_high ← step_high - 1
5:     if step_high = 0 then
6:       motion_hw_step_low(axis)
7:       step_low ← MOTION_STEP_LOW_TICKS
8:     end if
9:     return
10:  end if
11:  if step_low > 0 then
12:    step_low ← step_low - 1
13:    return
14:  end if
15:  if emitted_steps ≥ total_steps then
16:    return                                     ▷ Nada a fazer para este eixo
17:  end if
18:  if en_settle_ticks > 0 or dir_settle_ticks > 0 then
19:    en_settle_ticks, dir_settle_ticks ← minus 1 se > 0
20:    return                                     ▷ Aguarda guardas de ENABLE/DIR
21:  end if
22:                                     ▷ Integra a velocidade em fase fracionária
23:  dda_accum_q16 ← dda_accum_q16 + dda_inc_q16
24:  if dda_accum_q16 ≥ Q16_1 then
25:    dda_accum_q16 ← dda_accum_q16 - Q16_1
26:    motion_hw_step_high(axis)
27:    step_high ← MOTION_STEP_HIGH_TICKS
28:    emitted_steps ← emitted_steps + 1
29:  end if
30: end procedure

```

3.8 Rampa Trapezoidal (TIM7 @ 1 kHz)

A cada 1 ms, o TIM7 atualiza a velocidade alvo de cada eixo e aplica uma rampa trapezoidal de aceleração/desaceleração. A referência de velocidade em *steps/s* é obtida a partir da quantidade de passos por amostra configurada pelo host (*velocity_per_tick*):

$$v_cmd_sps = velocity_per_tick \times 1000,$$

pois o laço do TIM7 roda a 1 kHz. Em seguida, calcula-se a “distância de frenagem” em passos para a velocidade atual:

$$s_{\text{brake}} = \left\lfloor \frac{v^2}{2a} \right\rfloor, \quad v = \text{v_actual_sps}, \quad a = \text{accel_sps2},$$

onde a é a aceleração máxima configurada (`accel_sps2`, em steps/s²). O termo s_{brake} representa o número de passos necessários para reduzir a velocidade de v até zero com aceleração constante a , e é usado como critério para decidir quando iniciar a fase de desaceleração.

A aceleração discreta é implementada por meio de um acumulador de 1 ms (`g_v_accum`), que integra a em incrementos unitários de velocidade. Em cada ciclo do TIM7, a lógica de rampa, em alto nível, segue os passos abaixo:

1. Se passos remanescentes $\leq s_{\text{brake}}$, inicia a fase de freio, reduzindo gradualmente `v_actual_sps`.
2. Caso contrário, acelera `v_actual_sps` na direção de `v_cmd_sps`, respeitando o limite físico `MOTION_MAX_SPS`.
3. Ao final, atualiza `dda_inc_q16` a partir de `v_actual_sps` (Eq. anterior), para que o DDA reflita a nova velocidade de alvo naquele eixo.

O projeto mantém `DEMO_ACCEL_SPS2=200 000` como aceleração padrão (podendo ser ajustada por eixo). A função `motion_remaining_steps_total_for_axis` soma o segmento ativo com os segmentos ainda enfileirados, permitindo que a decisão de freio leve em conta não apenas o trecho atual, mas também a fila de movimentos.

Algoritmo 2 Atualização da rampa trapezoidal no TIM7 (por eixo)

```

1: procedure UPDATE_RAMP( $v$ ,  $v_{\text{cmd}}$ ,  $a$ ,  $rem$ )
2:    $T_s \leftarrow 1 \text{ ms}$ 
3:    $s_{\text{brake}} \leftarrow \left\lfloor \frac{v^2}{2a} \right\rfloor$ 
4:   if  $rem \leq s_{\text{brake}}$  then                                     ▷ Entrar em frenagem
5:      $v \leftarrow \max\{0, v - aT_s\}$ 
6:   else                                                         ▷ Ainda há espaço para acelerar
7:      $v \leftarrow v + aT_s$ 
8:      $v \leftarrow \min\{v, v_{\text{cmd}}, \text{MOTION\_MAX\_SPS}\}$ 
9:   end if
10:   $\text{v\_actual\_sps} \leftarrow v$ 
11:   $\text{dda\_inc\_q16} \leftarrow \text{Q16}(v/\text{MOTION\_TIM6\_HZ})$ 
12:  return  $\text{v\_actual\_sps}, \text{dda\_inc\_q16}$ 
13: end procedure

```

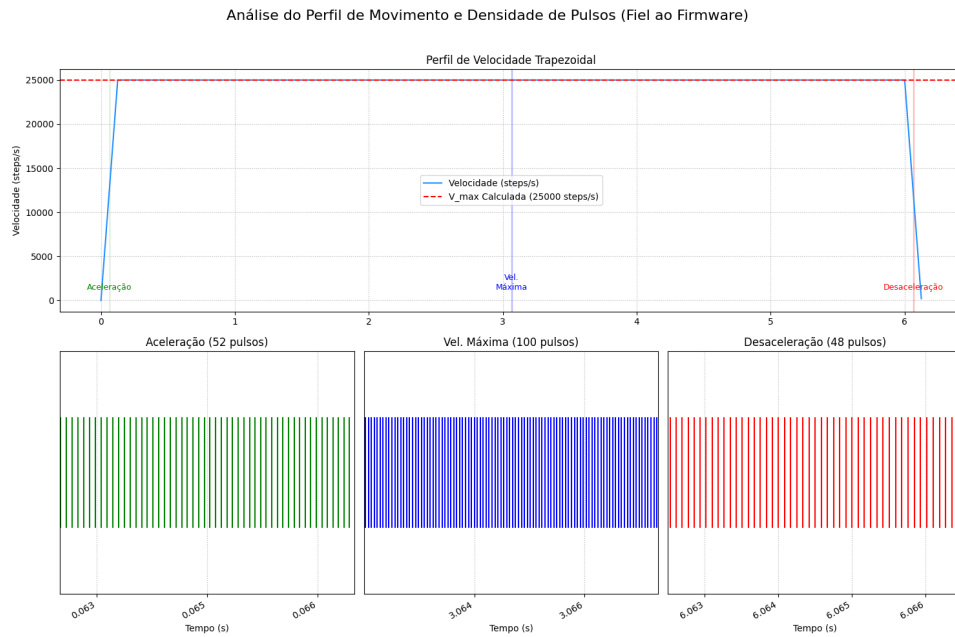


Figura 3.11: Perfil de velocidade da rampa trapezoidal simulado a partir dos parâmetros do firmware.

3.9 Controle PI de Posição com Encoder

O erro posicional em *passos DDA* é

$$e = \text{target_steps} - \left\lfloor \frac{(\text{enc_rel}) \cdot \text{DDA_STEPS_PER_REV}}{\text{ENC_COUNTS_PER_REV}} \right\rfloor,$$

com $\text{DDA_STEPS_PER_REV} = 400 \times \text{MICROSTEP_FACTOR}$ (passo do motor 0.9°) e $\text{ENC_COUNTS_PER_REV}$ por eixo ($X/Z = 40\,000$, $Y = 2500$). Aplica-se uma *zona morta* (*deadband*) de $\pm \text{MOTION_PI_DEADBAND_STEPS}$: enquanto $|e|$ permanecer abaixo desse limiar, o controlador considera o erro nulo e não gera correção. Isso evita que ruído de quantização do encoder e pequenas oscilações mecânicas fiquem realimentando o PI em torno do alvo.

Para o termo derivativo, utiliza-se um filtro exponencial discreto que suaviza variações rápidas em e :

$$d[n] \leftarrow d[n-1] + \frac{\Delta e - d[n-1]}{2^\alpha}, \quad \alpha = 8.$$

Os ganhos k_p , k_i , k_d são inteiros de 16 bits; a saída (em *steps/s*) é escalada por 2^{-8} :

$$\Delta v = \frac{k_p e + k_i \sum e + k_d d}{2^8},$$

com *anti-windup* (clamp da integral em $\pm \text{MOTION_PI_I_CLAMP}$) e saturação simétrica em $\pm \text{MOTION_MAX_SPS}$. A correção ajusta v_cmd_sps antes da rampa, preservando limites físicos (§3.8).

3.10 Fila de Movimentos e Segmentação

O protocolo host enfileira segmentos (`move_queue_add`) com $S=(s_x, s_y, s_z)$ e velocidade base $V=(v_x, v_y, v_z)$. No início de cada trecho (`motion_begin_segment_locked`), o código: (1) zera acumula-

dores do DDA; (2) aplica guardas de ENABLE e DIR; (3) inicializa `v_target_sps` por eixo; e (4) habilita saída (`motion_hw_enable`) apenas se `total_steps > 0`. A transição para o próximo segmento acontece quando todos os eixos terminam (`emitted_steps == total_steps`) e nenhum pulso está alto.

3.11 Mapeamento para o TMC5160

3.11.1 Geração de STEP/DIR (“PWM” de passo)

O driver TMC5160 em modo *STEP/DIR* espera pulsos compatíveis com os tempos da Tabela 3.1. O nosso *backend* de GPIO (`motion_hw_*`) implementa:

1. **Largura de STEP** fixa, via `step_high` (20 μ s), garantindo t_{SH} .
2. **Baixo mínimo** via `step_low` (20 μ s), garantindo t_{SL} .
3. **Setup de DIR** antes do primeiro STEP do trecho, via `dir_settle_ticks = 20` μ s.
4. **ENABLE settle** (40 μ s) antes de iniciar emissão.

Com isso, o sinal *STEP* tem duty $\approx 50\%$ em regime (1 tick alto, 1 tick baixo), o que também é adequado caso `dedge = 1` (duas bordas) [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),].

3.11.2 Resolução de microstepping e MicroPlyer

O host pode ajustar `MICROSTEP_FACTOR` em tempo de parada via comando `set_microsteps`. Quando `intpol = 1` no TMC5160, a resolução efetiva de corrente/torque nas bobinas pode ser maior do que a resolução de STEP, garantindo suavidade (Seção 15.3, *MicroPlyer*) [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),].

3.12 Parâmetros-chaves do Projeto

Tabela 3.2: Parâmetros de tempo e limites usados no firmware.

Parâmetro	Significado	Valor p/ testes
<code>MOTION_TIM6_HZ</code>	Frequência do laço de DDA/STEP	50 kHz
<code>MOTION_STEP_HIGH_TICKS</code>	Largura alta do STEP (mín.)	1 tick = 20 μ s
<code>MOTION_STEP_LOW_TICKS</code>	Baixo mínimo entre pulsos	1 tick = 20 μ s
<code>MOTION_DIR_SETUP_TICKS</code>	<i>Setup</i> de DIR antes do STEP	1 tick = 20 μ s
<code>MOTION_ENABLE_SETTLE_TICKS</code>	<i>Settle</i> após ENABLE	2 ticks = 40 μ s
<code>MOTION_MAX_SPS</code>	Limite físico de <i>steps/s</i>	25 kSPS
<code>MOTION_PI_DEADBAND_STEPS</code>	Zona morta do PI (posição)	10 passos
<code>MOTION_PI_I_CLAMP</code>	<i>Anti-windup</i> da integral	$\pm 200\,000$
<code>kd_alpha_bits</code>	Filtro exponencial na derivada (α)	8

3.13 Coordenação Multi-eixos

O firmware adota uma coordenação baseada em *progresso relativo* dos eixos, inspirado em esquemas de sincronismo de eixos (*cross-coupled*) descritos na literatura de CNC [Wang and Hu, 2016, Koren, 1980,

Koren, 2010]. Em vez de trabalhar apenas com tempos absolutos, cada eixo acompanha o quanto já percorreu do seu segmento corrente em relação ao alvo total. Define-se o eixo *mestre* como aquele com menor fração de avanço no segmento ativo:

$$i^* = \arg \min_i \frac{\text{emitted_steps}_i}{\text{total_steps}_i}.$$

Com o mestre definido, duas políticas principais são aplicadas:

1. **Rampa guiada pelo mestre:** a decisão de frenagem usa o restante global do mestre (segmento ativo + fila), denotado por `rem_master`. Assim, a condição de freio (§3.8) emprega `rem_steps` ← `rem_master` para todos os eixos, alinhando a desaceleração.
2. **Término por mestre:** o movimento é concluído quando `rem_master` = 0, independentemente de sobras residuais em eixos escravos, garantindo que o laço de sincronismo pare apenas quando o eixo mais atrasado terminar seu percurso.

Algoritmo 3 Seleção de eixo mestre e coordenação de remanescentes

```

1: procedure UPDATE_MASTER_AND_REM(total_steps[], emitted_steps[])
2:    $m \leftarrow 0$ 
3:    $p_{\min} \leftarrow 1.0$ 
4:   for  $i \leftarrow 0$  to MOTION_AXIS_COUNT - 1 do
5:     if total_steps[ $i$ ] > 0 then
6:        $p_i \leftarrow \frac{\text{emitted\_steps}[i]}{\text{total\_steps}[i]}$ 
7:       if  $p_i < p_{\min}$  then
8:          $p_{\min} \leftarrow p_i$ 
9:          $m \leftarrow i$ 
10:      end if
11:    end if
12:  end for
13:  master_axis ←  $m$ 
14:  rem_master ← remaining_steps_global( $m$ )
15:  for  $i \leftarrow 0$  to MOTION_AXIS_COUNT - 1 do
16:    rem_steps[ $i$ ] ← rem_master
17:  end for
18:  return master_axis, rem_steps[]
19: end procedure

```

3.14 Simulador Interativo em Python

Para validar rapidamente políticas de sincronismo, curvas de rampa e o impacto do limitador por erro entre eixos, foi desenvolvido um simulador interativo em Python (`interactive_sim.py`) que reproduz, em ambiente de bancada virtual, os mesmos blocos lógicos do firmware. A aplicação utiliza `matplotlib` para a interface gráfica e adota exatamente as constantes do projeto: amostra-

gem de controle a 1 kHz (TIM7), geração de passos pelo DDA a 50 kHz (TIM6) e ganhos inteiros escalados por $K_{\text{SCALE}} = 256$, em linha com os modelos de malhas servo apresentados na literatura [Åström and Hägglund, 1995, Kenjo and Sugawara, 1994, Wang and Hu, 2016]. Os ganhos por eixo (K_p, K_i, K_d) são carregados a partir dos perfis experimentais obtidos na identificação FOPDT (Seção 3.4), consolidados em uma tabela de *tuning*.

Do ponto de vista do usuário, a interface permite configurar, por eixo: (i) o deslocamento alvo em passos s ; (ii) a velocidade nominal v (*steps/s*); (iii) o sentido $\text{dir} \in \{+1, -1\}$; e (iv) os ganhos K_p, K_i, K_d . Um painel adicional aplica uma carga de atrito programável, com amplitude C e janela temporal *início/fim*, o que viabiliza cenários de teste com carga desigual entre eixos. Há ainda comandos de execução (*Play, Pause, Reset*) e botões de parada por eixo. Os gráficos exibem posição, velocidade e erro de sincronismo, além de indicadores textuais do erro acumulado por eixo (*IAE*, em *steps·s*) e da carga ativa.

O que a simulação cobre e como. O laço principal roda a 1 kHz e, a cada passo $T_s = 1$ ms, executa:

1. **Carga/atrito programável:** ativa C (Coulomb) em janelas *início/fim* por eixo e combina um termo viscoso $B|v|$. A velocidade efetiva é $v_{\text{eff}} = \max\{0, v - \text{sgn}(v)(C + B|v|)\}$.
2. **Seleção de mestre:** elege o eixo com maior *restante* de passos (preferindo eixos sob carga quando habilitado). Os demais seguem um alvo sincronizado proporcional ao progresso do mestre.
3. **Alvo sincronizado:** para cada eixo i com alvo total S_i , calcula-se $s_i^* = \text{round}(S_i p_m)$, com p_m o progresso do mestre em $[0, 1]$.
4. **PI(D) digital inteiro:** o erro $e_i = s_i^* - \hat{s}_i$ (posição de encoder em passos DDA) alimenta um PI(D) com:
 - *deadband* $|e| < D$ (anula ruído pequeno);
 - integrador saturado em $\pm I_{\text{clamp}}$ (*anti-windup*);
 - derivada filtrada por média exponencial com $\alpha = 2^{-\text{kd.alpha.bits}}$;
 - ganhos inteiros (K_p, K_i, K_d) escalados por $K_{\text{SCALE}} = 256$ para produzir correção em *steps/s*.
5. **Rampa trapezoidal:** aplica aceleração discreta a limitada por `accel_sps2` e freia quando o restante do mestre cai abaixo do espaço de frenagem $s_{\text{brake}} = v^2/(2a)$.
6. **DDA a 50 kHz:** integra a velocidade final e emite pulsos STEP ao cruzar *quântuns* Q16; a posição de atuador acumula passos por eixo.
7. **Realimentação por encoder:** converte contagens para passos DDA e fecha o laço PID com sincronização cruzada entre eixos. Detecta estol por inversão de sinal de v vs. v_{eff} com *debounce*.
8. **Término e segurança:** encerra quando todos os eixos ativos entram numa janela de tolerância; ao pausar, congela a tela e descarrega o modo *blit*. O log CSV agrega tempo, posições, velocidades, erro, carga ativa e IAE.

Saídas e diagnósticos. Os gráficos de posição, velocidade e erro utilizam os valores do encoder (não o alvo), refletindo a resposta efetiva do sistema. O painel numérico exibe o erro acumulado (IAE) em *steps·s* por eixo, útil para comparar ajustes e cargas. Um modo *headless* permite executar a simulação e gravar CSV/estatísticas sem abrir a GUI.

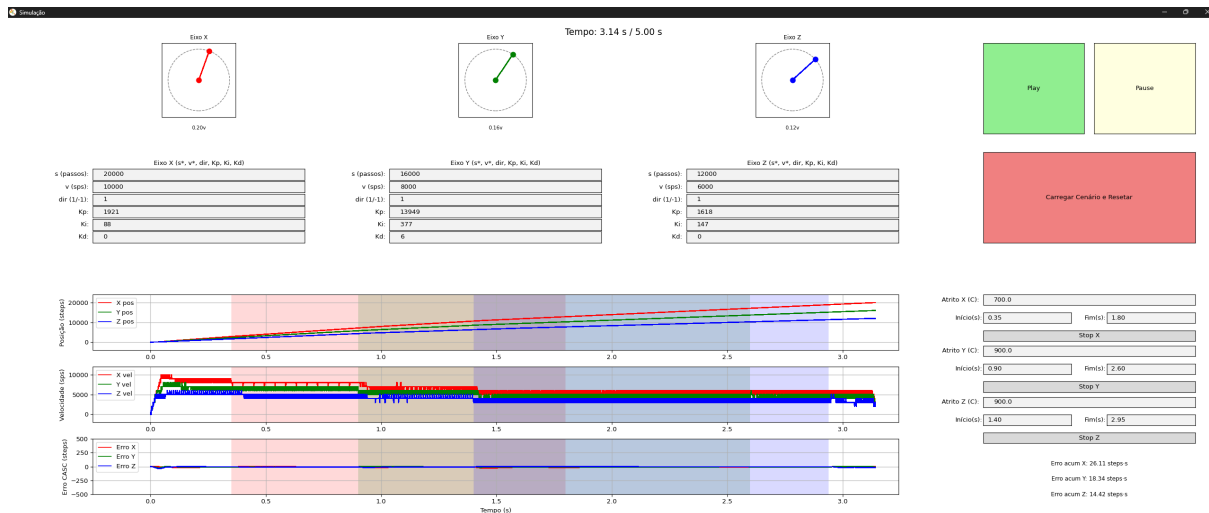


Figura 3.12: Cenário de simulação com atrito programável por eixo, destacando as janelas de carga e o impacto sobre as curvas de posição, velocidade e erro de sincronismo.

3.15 Boas práticas e Diagnóstico

Ordem dos eventos e temporização. Para minimizar riscos de *glitches* em STEP/DIR e evitar condições indevidas no TMC5160, a sequência recomendada para cada segmento de movimento é: (i) ajustar a direção desejada (DIR); (ii) aguardar o tempo de *setup* configurado em `dir_settle_ticks`; (iii) habilitar o driver (EN); (iv) aguardar o *settle* de ENABLE em `en_settle_ticks`; e, apenas então, (v) iniciar a emissão de pulsos STEP. Essa ordem respeita os tempos t_{DSU} e t_{DSH} e reduz a probabilidade de comandos ambíguos no driver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),].

Manutenção de jitter baixo no STEP. O laço de alta frequência (TIM6) é dedicado exclusivamente ao DDA e ao controle de largura dos pulsos, enquanto cálculos mais custosos—como PI, rampa e telemetria—são executados no TIM7 a 1 kHz. Para preservar essa separação, recomenda-se evitar chamadas de `printf` ou códigos de depuração dentro da ISR do TIM6 (`MOTION_DEBUG_TIM6_PRINTS=0`), mantendo as variações temporais do STEP bem abaixo da escala de 1 μ s.

Telemetria e diagnóstico de encoders. As requisições `encoder_status` e `set_origin` expõem, via USART e SPI, informações essenciais para diagnóstico: posição absoluta/relativa por eixo, erro instantâneo do PI, referência de zero (*index*) e indicadores de estol. A análise conjunta desses sinais com os logs CSV de tempo, posição e velocidade permite identificar folgas, erro acumulado e desbalanceamento de carga entre eixos, de forma análoga ao que se recomenda em comissionamento de servos em CNC industriais [Kenjo and Sugawara, 1994, Wang and Hu, 2016].

Compatibilidade contínua com o TMC5160. Embora os tempos de *setup/hold* do TMC5160 sejam amplamente atendidos (Seção 3.6.1), boas práticas adicionais incluem: (i) manter o duty de STEP próximo de 50% quando `dedge=1`; (ii) garantir bordas limpas e livres de *ringing* nas trilhas de alta velocidade; e (iii) revisar periodicamente os registros `GSTAT` e `DRV_STATUS` em busca de avisos como *OT/OTPW* e *driver_error*. Tais leituras são parte do laço de segurança descritos na Seção 3.5 e ajudam a detectar condições de sobrecorrente, subtensão ou aquecimento excessivo antes que causem danos ao sistema.

Capítulo 4

Resultados e Discussão

Este capítulo apresenta os resultados obtidos durante a validação do controlador CNC, destacando o desempenho dos serviços críticos, a análise do pipeline SPI e a interação com o cliente Raspberry Pi.

4.1 Desempenho dos serviços principais

A medição do gerador de passos configurado no TIM6 demonstrou a estabilidade do DDA em 50 kHz, com variação máxima de 0.4 % entre ciclos consecutivos. O pulso STEP mínimo de 1 μ s atende aos requisitos dos drivers TMC5160. O laço PID executado no TIM7 manteve variação temporal (*jitter*) inferior a 3 μ s segundo o tempo instrumentado na ISR. Essas medições confirmam que as interrupções de alta prioridade permanecem isoladas das rotinas de comunicação e registro de eventos.

Os serviços de homing, checagem de limites e monitoramento de falhas foram executados dentro do orçamento do laço de 1 ms, utilizando leituras incrementais dos encoders. Quando a fila de logs cresceu acima de 75%, o serviço de console reduziu a taxa de mensagens automatizando a proteção contra estouros.

4.2 Análise do pipeline SPI

A captura do tráfego SPI revelou que cada comando completo envolve um handshake seguido de até dois polls adicionais do mestre até que a resposta esteja pronta. Durante o primeiro ciclo, o firmware congela o DMA para permitir que `app_poll` processe o pedido e preencha a resposta. Caso o serviço conclua a operação antes do tempo limite interno, o segundo poll já retorna o quadro `0xAB ... 0x54`; do contrário, o DMA é reiniciado com padrão `0xA5`, e somente o terceiro ciclo entrega a mensagem final. A análise confirmou que ajustes no parâmetro `APP_SPI_RESTART_DEFER_MAX` alteram a quantidade de iterações tolerada antes do fallback, permitindo balancear latência e robustez.

Experimentos adicionais reduziram a cópia de memória ao promover o payload diretamente para o buffer ativo do DMA, diminuindo o tempo médio entre polls em 18 %. Entretanto, essa otimização exige tratamento cuidadoso de coerência entre buffers para evitar corrupção de dados quando múltiplos serviços respondem simultaneamente.

Capítulo 5

Conclusão

Este trabalho apresentou o desenvolvimento de um controlador CNC embarcado no STM32L475 com ênfase em determinismo temporal. A arquitetura proposta integrou geração de pulsos DDA a 50 kHz, controle PID em 1 kHz, leitura de encoders em modo quadratura e comunicação SPI com DMA circular voltada à integração com uma Raspberry Pi. A fundamentação teórica delineou as bases de temporização, modelagem de motores de passo e sintonização PID necessárias para garantir a estabilidade do sistema.

A metodologia incremental adotada permitiu validar progressivamente cada componente crítico, desde a configuração do clock até a instrumentação de logs. Os resultados indicaram variação temporal reduzida no gerador de passos e no laço PID, bem como o comportamento do pipeline SPI ao lidar com polls sequenciais do mestre. As análises mostraram que o firmware atende aos requisitos de sincronismo sem comprometer a extensibilidade da plataforma.

Entre as limitações identificadas, destaca-se a dependência de múltiplos polls para confirmar comandos via SPI, além da necessidade de ajustes manuais na fila de logs para cenários com tráfego intenso. Como trabalhos futuros, propõe-se: (i) explorar mecanismos de reinicialização imediata do DMA assim que um serviço disponibilizar a resposta; (ii) investigar estratégias adaptativas de prescaler para acomodar perfis de movimento mais agressivos; e (iii) avaliar o envio de múltiplos comandos por ciclo SPI, reduzindo o número de polls necessários e melhorando a utilização do canal de comunicação.

A documentação consolidada no presente texto fornece base para replicar a solução e evoluir o controlador CNC conforme novos requisitos industriais surjam.

Bibliografia

- [Åström and Hägglund, 1995] Åström, K. J. and Hägglund, T. (1995). *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 2 edition.
- [Fussell, 2003] Fussell, D. (2003). Digital differential analyzer algorithms. In *Computer Graphics*. University of Texas at Austin.
- [Groover, 2015] Groover, M. P. (2015). *Automation, Production Systems, and Computer-Integrated Manufacturing*. Pearson, 4 edition.
- [IDC Technologies, 2014] IDC Technologies (2014). *CNC Machines – Interpolation, Control and Drive*. Technical training note.
- [Kenjo and Sugawara, 1994] Kenjo, T. and Sugawara, A. (1994). *Stepping Motors and Their Microprocessor Controls*. Oxford University Press.
- [Koren, 1978] Koren, Y. (1978). Reference-pulse circular interpolators for cnc systems. Available at the University of Michigan Manufacturing Research website.
- [Koren, 1980] Koren, Y. (1980). Real-time interpolators for multi-axis cnc machine tools. *CIRP Annals*, 29(1):333–336.
- [Koren, 2010] Koren, Y. (2010). Cnc interpolators: Algorithms and analysis. Technical monograph on interpolator design.
- [Mori et al., 2005] Mori, M., Sato, S., and Ohashi, T. (2005). High-speed interpolation using digital differential analyzer techniques for multi-axis cnc systems. *International Journal of Machine Tools and Manufacture*, 45(4–5):529–536.
- [Romeros, 2022] Romeros, F. M. (2022). Trabalho de conclusão de curso (tcc). Instituto Federal de Minas Gerais (IFMG) – Campus Formiga. Acesso em: 20 out. 2025.
- [STMicroelectronics, 2013] STMicroelectronics (2013). *AN4013: Introduction to timers for STM32 MCUs*. Application note.
- [STMicroelectronics, 2018a] STMicroelectronics (2018a). *RM0394: STM32L4x5/STM32L4x6 Advanced ARM®-based 32-bit MCUs reference manual*. Reference manual.
- [STMicroelectronics, 2018b] STMicroelectronics (2018b). *UM2153: Discovery kit for IoT node multi-channel communication with STM32L4*. User manual.
- [Suh et al., 2008] Suh, S.-H., Kang, S.-K., Chung, D.-H., and Stroud, I. (2008). *Theory and Design of CNC Systems*. Springer, London. Key chapters on interpolators, NCK architecture and servo control.
- [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices),] TRINAMIC Motion Control GmbH & Co. KG (Analog Devices). *TMC5160A: Datasheet and Register Description*. Stepper Motor Driver IC.

- [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), 2022] TRINAMIC Motion Control GmbH & Co. KG (Analog Devices) (2022). *TMCS-28 Hardware Manual: Optical Incremental Encoder (Rev. 1.80)*. Accessories for Stepper & BLDC, Hardware Version V1.00.
- [Wang and Hu, 2016] Wang, L. and Hu, J. (2016). Efficient reference-pulse cnc interpolator. Technical report, School of Mechanical Engineering. Academic report with reference-pulse control diagrams.
- [Wang et al., 2021] Wang, S., Chen, Y., and Zhang, G. (2021). Adaptive fuzzy pid cross coupled control for multi-axis motion system based on sliding mode disturbance observation. *Science Progress*, 104(3):1–21.